

머신러닝 9 교시

— 다중회귀분석 —

학습 목표

- 다중선형회귀 모형의 구조와 회귀계수의 의미를 설명할 수 있다.
- 회귀계수를 추정·구현할 수 있다.
- 변수의 선택방법 4 가지가 있음을 안다.
- 전진·후진·단계적 선택법으로 **최적 변수 집합을 선정**할 수 있다.
- VIF 를 계산해 **다중공선성을 진단**하고 변수를 제거하는 대응책을 제시할 수 있다
- Ames 주택 데이터에 다중회귀를 구현하고 시사점을 도출할 수 있다.

1. 다중선형회귀의 개념

단순선형회귀에서는 설명변수가 단 하나뿐이라 그릴 수 있는 그림이 오로지 직선이었다. 그러나 현실의 문제는 주택 가격 하나를 예측하기 위해서도 전반적인 마감 품질, 거주 면적, 차고 크기, 지하실 면적, 건축 연도처럼 여러 요인이 동시에 작용한다. 변수를 하나만 쓰면 적합도가 낮을 수밖에 없고, 엉뚱한 결론을 내리기 쉽다. 다중선형회귀는 여러 설명변수를 한꺼번에 모형에 넣어 각 변수가 종속변수에 기여하는 정도를 동시에 추정한다.

1.1 회귀모형과 목표

핵심 정리

- 단순선형회귀: $y = \beta_0 + \beta_1x + \epsilon$ (직선)
- 다중선형회귀: $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_px_p + \epsilon$ (초평면)
- 설명변수가 p 개로 늘면 적합되는 도형은 직선 $\rightarrow$ 평면 $\rightarrow$ 초평면으로 확장된다.
- 목표: 잔차 제곱합을 최소화하는 $\beta_0, \beta_1, \dots, \beta_p$ 를 찾는 것.

다중선형회귀의 모형식은 단순선형회귀에 변수만 더 붙인 형태이다. 핵심은 우변이 여전히 회귀 계수에 대해 "선형"이라는 점이다. x_1^2 이나 $x_1 \cdot x_2$ 같은 비선형 항이 들어와도, 그 항을 새로운 변수 z 로 치환하면 모형은 여전히 z 에 대해 선형이다. 따라서 "선형 모형"이라는 말은 변수의 형태가 아니라 계수에 대한 선형성을 의미한다.

종속 변수와 설명 변수 선택 방법

- ☑ 관심 있는 데이터에 대한 선행연구
- ☑ 이론적인 배경
- ☑ 분석가의 직관

위 사항들을 종합적으로 고려하여

종속 변수 Y 와 설명 변수 $X_1, X_2, \dots, X_p$ 들을 결정

회귀모형

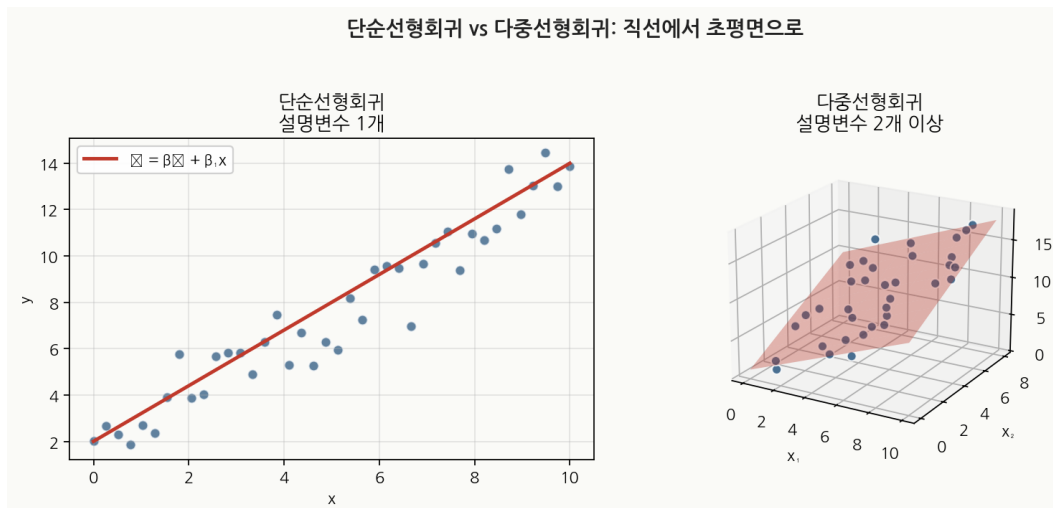
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_d x_d + \varepsilon$$



종속 변수와 설명 변수들의 관계를 분석

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_d x_d$$

[그림 1] 다중선형회귀 모형의 일반식과 종속·설명 변수 선택 절차



[그림 2] 단순선형회귀는 직선, 다중선형회귀는 초평면으로 데이터에 적합한다

1.2 회귀계수의 의미

다중회귀의 회귀계수 β_j 는 "다른 변수가 모두 고정되어 있을 때, x_j 가 한 단위 증가하면 y 가 평균적으로 얼마만큼 변하는가"를 나타낸다. 이것을 편회귀계수(partial regression coefficient)라고 부른다. 단순회귀의 기울기와 이름은 비슷하지만 해석에 "다른 변수가 고정된 상태"라는 조건이 따라붙는다는 점이 결정적으로 다르다.

예컨대 Ames 데이터에서 주택 가격(SalePrice)을 거주 면적(GrLivArea)과 전반 품질(OverallQual)로 회귀해 $\beta_{\text{GrLivArea}} = 58.8$ (달러/sqft)라는 결과를 얻었다면, 이는 "OverallQual이 같은 등급일 때 거주 면적이 1 sqft 더 넓으면 가격이 평균 약 58.8 달러 더 비싸다"는 의미이다. OverallQual을 고려하지 않은 단순회귀의 기울기와는 보통 값이 다르게 나온다 — 단순회귀에서는 면적과 함께 변하는 다른 품질 요인까지 기울기 안에 섞여 들어오기 때문이다.

1.3 회귀계수 추정: 최소제곱법(OLS)

회귀계수를 어떻게 정할 것인가. 가장 널리 쓰이는 방법이 최소제곱법(OLS, Ordinary Least Squares)이다. 관측값 y_i 와 예측값 $\hat{y}_i$ 의 차이를 잔차라 하고, 잔차의 제곱합을 가장 작게 만드는 계수를 고른다.



회귀 계수의 추정

📖 (참고) 최소제곱법(Ordinary Least Squares : OLS)

행렬을 이용해서 앞의 과정을 다시 쓰면 아래와 같다.

$$\text{예측할 } y \text{ 값: } \hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_d x_d$$

$$\min \sum_{i=1}^n (y_i - \hat{y}_i)^2 = (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \dots + \hat{\beta}_d x_{id}))^2$$

⇒ 주어진 데이터 y, X 에 대해 $\min(y - X\beta)^T(y - X\beta)$ 인 β 를 찾는 것

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

(단순선형회귀분석과 동일한 방식으로 회귀계수 추정 가능)

[그림 3] 최소제곱법(OLS)으로 다중회귀 계수 β 를 행렬식 한 줄로 추정한다

이 최적화 문제는 다행히 닫힌 해(closed-form solution)가 존재한다. 설계행렬 X 에 절편 열을 포함하고 종속변수를 벡터 y 로 두면 OLS 추정량 $\hat{\beta}$ 은 $(X^T X)^{-1} X^T y$ 로 표현된다. 단순회귀와 동일한 원리이지만 변수가 여러 개가 되었기에 행렬 연산으로 한 번에 풀어낸다는 점만 다르다.

함정 / 주의

- $(X^T X)$ 가 역행렬을 가지려면 설명변수들이 서로 완전한 선형종속이 아니어야 한다.
- 변수끼리 너무 상관이 높으면(다중공선성) $(X^T X)$ 가 거의 특이행렬에 가까워져 계수 추정이 불안정해진다.
- 실무에서는 sklearn LinearRegression 이 내부적으로 SVD 기반 의사역행렬을 사용해 이 위험을 줄인다.

1.4 단순 예제: iris 데이터

iris 데이터셋에서 sepal length 를 반응변수(y)으로 하고 다른 3 개 변수를 설명변수로 하여 각 변수의 계수를 구하는 다중회귀 분석을 해 보자.

scikit-learn 사용

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# 1) 데이터
iris = load_iris(as_frame=True).frame.drop(columns=['target'])
y = iris['sepal length (cm)']
X = iris.drop(columns=['sepal length (cm)'])

# 2) 분할
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3) 학습 / 예측
lr = LinearRegression().fit(X_train, y_train)
y_pred = lr.predict(X_test)

# 4) 평가
print(f"R^2 : {r2_score(y_test, y_pred):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")

# 5) 계수 / 절편
for name, coef in zip(X.columns, lr.coef_):
    print(f"{name:25s} {coef:+.4f}")
print(f"Intercept: {lr.intercept_:.4f}")
```

분석 결과 :

```
R^2 : 0.8526
RMSE: 0.3132
sepal width (cm)      +0.6634
petal length (cm)     +0.7588
petal width (cm)      -0.6883
Intercept: 1.7624
```

statsmodels 사용

```
import numpy as np
import statsmodels.api as sm
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

# 1) 데이터
iris = load_iris(as_frame=True).frame.drop(columns=['target'])
y = iris['sepal length (cm)']
X = iris.drop(columns=['sepal length (cm)'])
```

```

# 2) 분할
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3) 상수항 추가 (statsmodels 는 절편을 자동으로 넣지 않음)
X_train_c = sm.add_constant(X_train)
X_test_c = sm.add_constant(X_test)

# 4) 학습 / 예측
model = sm.OLS(y_train, X_train_c).fit()
y_pred = model.predict(X_test_c)

# 5) 평가 (테스트셋 기준)
print(f"Test R^2 : {r2_score(y_test, y_pred):.4f}")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
print()

# 6) 전체 통계 요약 (계수, p 값, 신뢰구간, F 검정 등)
print(model.summary())

```

statsmodels 는 모델에 데이터를 넣는 방식이 두가지가 있다.

```

import statsmodels.api as sm
import statsmodels.formula.api as smf

```

smf 를 사용하면 더 쉬운 문법을 쓸 수 있다. smf.ols 는 스타일 수식 문자열을 받고, 데이터는 종속변수와 독립변수가 **하나의 데이터프레임**에 같이 있으면 된다. 따라서 X, y 데이터셋을 분리할 필요가 없다. 또, smf.ols 는 y 절편을 자동으로 넣어준다. 그래서 sm.add_constant 로 상수항을 직접 추가할 필요가 없다.

1.4 실전 예제: Ames Housing 데이터

Ames Housing Dataset 은 미국 아이오와 주 Ames 시의 1,460 채 주택에 대해 약 80 개의 특성을 기록한 데이터셋으로, Dean De Cock 교수가 2011 년 Journal of Statistics Education 에 발표하면서 다중회귀 학습용 표준 교재로 빠르게 자리잡았다. 데이터가 주택 한 채 단위로 세밀하고, 특성 수가 80 여 개로 풍부하며, 결측·범주형 처리 같은 실무 이슈가 자연스럽게 등장한다는 점에서 회귀 학습에 좋은 교재이다. 종속변수는 SalePrice(달러 단위 매매가) 이다. 본 교재에서는 회귀 시연에 적합한 14 개 수치 특성만 추려 사용한다.

Ames Housing — 본 강의에서 사용하는 14개 설명변수 + 1개 종속변수

OverallQual	주택 전반의 마감/재료 품질 (1=Very Poor ... 10=Very Excellent)
GrLivArea	지상 거주 면적 (Above-grade living area, sqft)
GarageCars	차고 수용 차량 수 (0~4)
TotalBsmtSF	지하실 전체 면적 (sqft)
YearBuilt	건축 연도 (1872~2010)
FullBath	지상층 욕실 수 (0~3)
LotArea	대지 면적 (sqft)
1stFlrSF	1층 면적 (sqft)
TotRmsAbvGrd	지상층 총 방 개수 (욕실 제외)
Fireplaces	벽난로 개수 (0~3)
OpenPorchSF	개방형 포치 면적 (sqft)
MasVnrArea	석조 외벽 면적 (sqft)
BedroomAbvGr	지상층 침실 수 (0~8)
HalfBath	지상층 반(half) 욕실 수 (0~2)
SalePrice ★	주택 매매가 (달러, 종속변수)

[그림 4] Ames Housing — 본 강의에서 사용하는 14 개 수치 설명변수 + 종속변수 SalePrice

이 14 개 특성에는 전반적인 마감 품질(OverallQual), 거주 면적(GrLivArea) 같은 직관적 변수와 함께 차고 수용 차량 수(GarageCars), 지하실 면적(TotalBsmtSF), 건축 연도(YearBuilt) 같은 주택의 물리적 속성, 그리고 대지 면적(LotArea), 벽난로 개수(Fireplaces), 석조 외벽 면적(MasVnrArea) 같은 부가 요소까지 골고루 들어 있다. 다중회귀의 "여러 요인을 동시에 보는" 가치가 잘 드러나는 구성이다.

Ames Housing 다중회귀 모형식 (의사코드)

$$\begin{aligned}
 \text{SalePrice} = & \beta_0 + \beta_1 \cdot \text{OverallQual} + \beta_2 \cdot \text{GrLivArea} + \beta_3 \cdot \text{GarageCars} \\
 & + \beta_4 \cdot \text{TotalBsmtSF} + \beta_5 \cdot \text{YearBuilt} + \beta_6 \cdot \text{FullBath} \\
 & + \beta_7 \cdot \text{LotArea} + \beta_8 \cdot \text{1stFlrSF} + \beta_9 \cdot \text{TotRmsAbvGrd} \\
 & + \beta_{10} \cdot \text{Fireplaces} + \beta_{11} \cdot \text{OpenPorchSF} + \beta_{12} \cdot \text{MasVnrArea} \\
 & + \beta_{13} \cdot \text{BedroomAbvGr} + \beta_{14} \cdot \text{HalfBath} + \epsilon
 \end{aligned}$$

1.5 적합 결과 읽기

회귀계수를 추정하고 나면 Python statsmodels 의 `OLS().summary()` 가 내놓는 표를 읽을 줄 알아야 한다. **계수(coef)**, **유의확률(p-value)**, 그리고 **결정계수 R²** 가 두 개로 나란히 출력된다는 점이다. 두 R² 중 어느 쪽을 봐야 할까?

OLS (Train: 871, 14)

$R^2 = 0.8020$ Adjusted $R^2 = 0.7988$
 F-stat = 247.66 p-value $\approx 2.3e-289$

	(coef)	std err	t	p-value	
OverallQual	+15,630	1,492	10.48	< 0.001	***
GrLivArea	+58.84	6.13	9.59	< 0.001	***
GarageCars	+9,978	2,320	4.30	< 0.001	***
TotalBsmntSF	+38.55	5.40	7.14	< 0.001	***
YearBuilt	+400.82	63.31	6.33	< 0.001	***
FullBath	-2,860	3,496	-0.82	0.414	
LotArea	+0.49	0.11	4.66	< 0.001	***
1stFlrSF	-5.17	6.77	-0.76	0.446	
TotRmsAbvGrd	+3,891	1,585	2.46	0.014	*
Fireplaces	+4,222	2,182	1.94	0.053	.
OpenPorchSF	+5.52	20.82	0.27	0.791	
MasVnrArea	+11.96	7.70	1.55	0.121	
BedroomAbvGr	-11,740	2,236	-5.25	< 0.001	***
HalfBath	-3,571	3,428	-1.04	0.298	

* p < 0.05 ** p < 0.01 *** p < 0.001

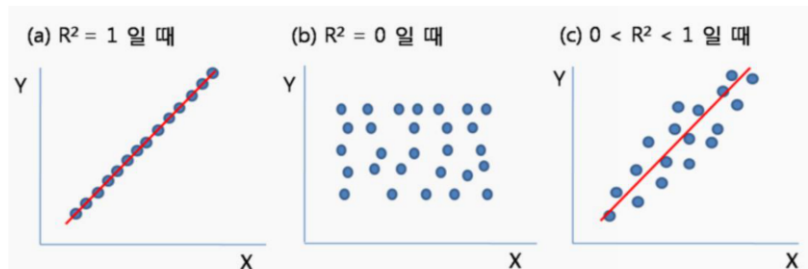
[그림 5] Ames 다중회귀의 OLS summary — 계수, 유의확률, R^2 · Adjusted R^2

2. 모형의 적합도 평가

모형을 만들었다면 그다음으로 따져야 할 것은 "이 모형이 데이터를 얼마나 잘 설명하는가"이다. 여기서 등장하는 대표 지표가 결정계수 R^2 와 수정 결정계수(Adjusted R^2)이다.

R^2 (결정계수): 반응변수(Y)의 전체 변동 중 설명변수(X)가 차지하는 변동의 비율

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}, \quad 0 \leq R^2 \leq 1 (\because SST=SSR+SSE)$$



설명변수 없이 항상 $\bar{y}$ 를 예측값으로 사용하는 모형과 회귀모형의 성능을 비교
 R^2 가 클수록 모형이 데이터를 잘 설명한다.

$R^2=1$: 모든 측정값들이 회귀직선 위에 있는 경우

$R^2=0$: 추정된 회귀직선은 X와 Y의 관계를 전혀 설명하지 못함

[그림 6] R^2 의 정의와 양 극단 — $R^2=1$ (완전 적합), $R^2=0$ (설명력 없음), 그 사이

2.1 R^2 (결정계수)

R^2 은 종속변수 y 의 전체 변동 가운데 회귀모형이 설명한 비율을 의미한다. 전체 변동은 평균을 기준으로 한 제곱합(SST)으로 측정하고, 회귀가 설명한 변동(SSR)과 잔차가 남긴 변동(SSE)으로 분해된다.

변동의 분해

$$SST = \sum_i (y_i - \bar{y})^2 \quad \# \text{총변동 (Total Sum of Squares)}$$

$$SSR = \sum_i (\hat{y}_i - \bar{y})^2 \quad \# \text{회귀가 설명한 변동 (Explained)}$$

$$SSE = \sum_i (y_i - \hat{y}_i)^2 \quad \# \text{잔차 변동 (Residual)}$$

$$SST = SSR + SSE$$

결정계수

$$R^2 = SSR / SST = 1 - SSE / SST, \quad 0 \leq R^2 \leq 1$$

R^2 이 1에 가까울수록 모형이 데이터의 변동을 거의 다 설명한다는 뜻이고, 0에 가까울수록 평균을 그냥 답으로 내놓는 단순한 예측기와 큰 차이가 없다는 뜻이다. 본 강의의 Ames 데이터에서 14개 변수를 모두 넣은 전체 모형은 훈련 데이터에서 $R^2 \approx 0.80$, 즉 SalePrice 변동의 약 80%를 모형이 설명하고 나머지 20%는 모형이 잡지 못한 부분이다.

2.2 수정 결정계수 (Adjusted R^2)

R^2 에는 결정적인 약점이 하나 있다. **변수를 아무리 쓸모없는 것이라도 추가하기만 하면 R^2 은 높아진다는 점이다.** 심지어 난수로 만든 변수를 넣어도 R^2 은 조금이라도 올라가는 경향이 있다. 이래서는 "변수를 많이 넣은 모형이 무조건 좋은 모형"이라는 잘못된 결론으로 이어진다. 수정 결정계수는 이 문제를 해결하기 위해 변수 개수 p 에 대한 패널티를 명시적으로 붙인다. 식을 보면 자유도($n - p - 1$)가 분모에 들어와, 변수를 추가하면 R^2 은 오르지만 ($n - p - 1$)은 줄어서 두 효과가 서로 상쇄된다. 새 변수가 정말로 설명력을 가질 때만 Adjusted R^2 이 올라간다. Ames 전체 모형의 경우 $R^2 = 0.802$, Adjusted $R^2 = 0.799$ 로 거의 차이가 없는데, 이는 14개 변수가 대체로 의미 있게 설명에 기여한다는 신호이다.

수정 결정계수

$$R^2_{\text{adj}} = 1 - (1 - R^2) \times (n - 1) / (n - p - 1)$$

의미

n : 표본 크기

p : 설명변수 개수 (절편 제외)

변수 p 를 늘리면 ($n - p - 1$)이 작아져 패널티가 커진다

따라서 $R^2_{\text{adj}} \leq R^2$ 이 항상 성립한다

Adjusted Coefficient of Determination

$$R^2_{adj} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

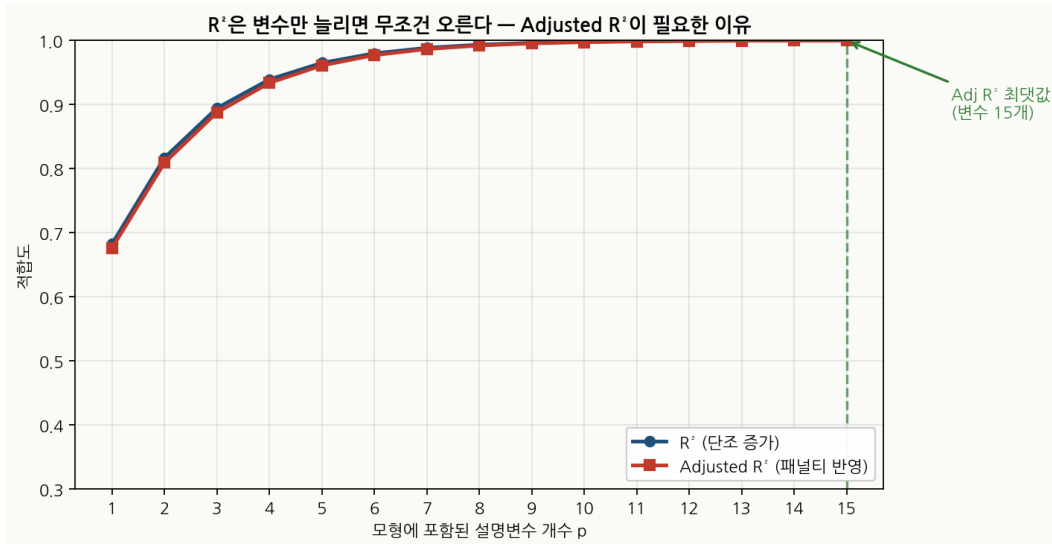
Variable Definitions

R^2_{adj} = Adjusted R^2 (수정 결정계수)

R^2 = Coefficient of Determination (결정계수)

n = Sample Size (샘플 크기)

p = Number of Predictors (예측 변수의 수)



[그림 7] 변수가 많아질수록 R²은 계속 오르지만 Adjusted R²은 어느 지점부터 꺾인다

2.3 모형 비교: 언제 어떤 기준을 쓰는가

두 지표는 쓰임이 다르다. R²은 "하나의 고정된 모형이 데이터를 얼마나 설명하는지"를 보여 주는 절대 적합도 지표이다. 반면 Adjusted R²은 "설명변수 개수가 서로 다른 여러 모형을 비교"할 때 쓰는 상대 비교 지표이다. 즉, 같은 데이터에 대해 변수 6개짜리 모형과 14개짜리 모형 중 어느 쪽이 더 "경제적으로" 잘 설명하는지를 가린다.

두 지표의 사용 가이드

- 모형 하나의 적합도만 보고 싶다 → R²
- 변수 개수가 다른 여러 모형 가운데 하나를 고르고 싶다 → Adjusted R² (또는 AIC, BIC)
- 새 변수를 넣었더니 R²은 올랐는데 Adjusted R²은 떨어졌다 → 그 변수는 빼는 것이 낫다는 신호

다음 3 장에서는 Adjusted R² 만이 아니라 AIC, BIC, Mallows' Cp 같은 다른 변수 선택 기준과 함께, 실제로 어떤 변수 조합을 모형에 남길지 결정하는 알고리즘(전진 선택법, 후진 소거법, 단계적 선택)을 다룬다.

3. 변수 선택 방법

2장에서 R^2 만 보면 변수를 마구 끼워 넣고 싶어진다는 사실을 확인했다. 그러나 변수가 늘어나면 모형은 복잡해지고, 해석은 어려워지며, 새 데이터에서의 예측 성능은 떨어지기 십상이다. 그래서 "설명변수 중에서 정말로 필요한 놈들만 추려내는" 작업이 필요하다. 이 작업을 변수 선택(variable selection) 또는 모형 선택(model selection)이라 부른다.

강의 슬라이드에서는 변수 선택 기법을 크게 세 갈래로 정리한다 — 가능한 모든 조합을 다 시도하는 전역 탐색(Exhaustive search), 한 번에 한 변수씩 더하거나 빼는 단계적 탐색(Stepwise selection), 그리고 계수 자체에 패널티를 줘서 자동으로 0에 가까운 변수를 끄게 만드는 LASSO 류이다. 본 강의에서는 처음 두 갈래(전역 + 단계적)를 깊이 다룬다.

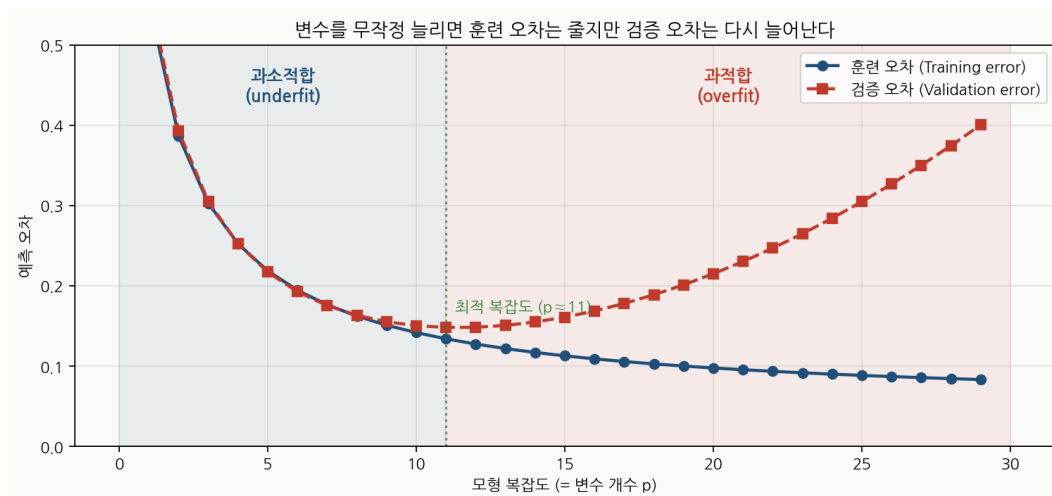
- 전역 탐색법(Exhaustive search)
- 단계적 탐색법(Stepwise Selection)
 - 전진 선택법(Forward Selection)
 - 후진 소거법(Backward Elimination)
 - Bidirectional Elimination
- LASSO(or Elasticnet)

[그림 8] 변수 선택 방법의 큰 분류 — 전역 탐색 / 단계적 탐색 / LASSO

3.1 변수 선택의 필요성

왜 굳이 변수를 골라야 하는가. 첫째로 모형 해석이다. 80 개 변수가 들어 있는 회귀식은 사람 머리로 읽기 어렵다. 6~10 개의 핵심 변수로 추려진 모형이라야 "어떤 요인이 가격을 얼마나 끌어올리는지" 한눈에 설명할 수 있다.

둘째로 일반화 성능이다. 변수가 많을수록 훈련 데이터에는 잘 맞춰지지만 새로운 데이터에서는 오히려 더 큰 오차를 낸다. 이른바 과적합(overfitting)이다. 모형 복잡도와 예측 오차의 전형적 관계는 다음과 같이 U 자 모양으로 나타난다.



[그림 9] 변수가 많아지면 훈련 오차는 줄지만 검증 오차는 다시 늘어난다

핵심 정리 — 왜 변수를 고르는가

- 해석성: 중요한 변수만 남기면 모형이 사람 말로 설명 가능해진다.
- 일반화: 잡음 변수를 빼야 새 데이터에서도 안정적인 예측이 가능하다.
- 비용: 운영 단계에서 수집해야 하는 변수가 적을수록 시스템이 가볍다.
- 목표: 최소한의 변수로 최대의 예측력을 내는 모형을 찾는 것.

3.2 모형 평가 기준 (AIC · BIC · Mallow's Cp · Adjusted R²)

변수를 더 넣을지 뺄지를 결정하려면 "현재 모형이 더 좋아졌는가"를 측정할 잣대가 필요하다.

단순한 R²은 변수를 늘리면 **절대 줄지 않으므로** 잣대로 부적합하다. 실무에서는 변수 개수에 **패널티를 명시적으로 붙인 네 가지** 기준을 흔히 사용한다.



변수 선택을 위한 모형 평가 기준

- AIC, BIC, 수정 R-제곱합, Mallow's Cp 등
- 위의 기준들은 모형 적합도와 변수의 개수를 동시에 고려한다.

[그림 10— AIC, BIC, 수정 R², Mallow's Cp]

네 가지 변수 선택 기준의 정의

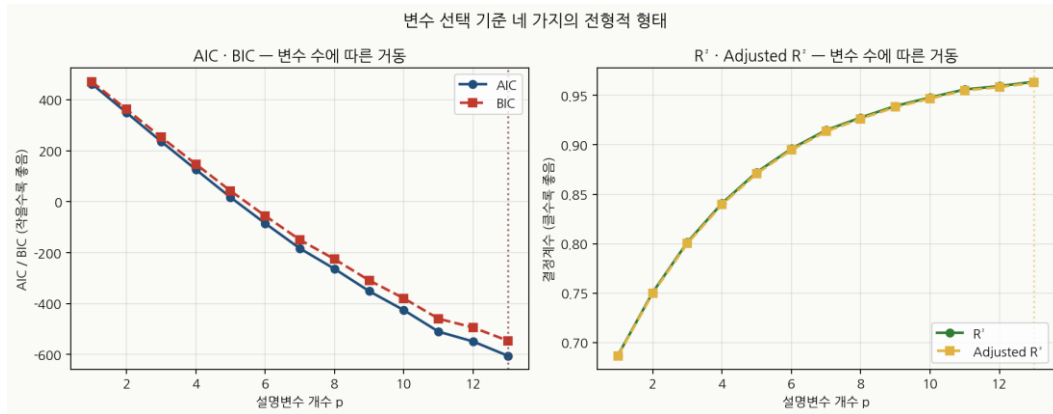
n: 표본 크기, p: 설명변수 개수, SSE: 잔차제곱합

σ^2_{hat} : 전체 모형에서 추정한 오차분산

$$\begin{aligned} \text{AIC} &= n * \ln(\text{SSE} / n) + 2 * (p + 1) \\ \text{BIC} &= n * \ln(\text{SSE} / n) + \ln(n) * (p + 1) \\ \text{Cp} &= \text{SSE} / \sigma^2_{\text{hat}} - n + 2 * (p + 1) \\ \text{R2adj} &= 1 - (1 - R^2) * (n - 1) / (n - p - 1) \end{aligned}$$

AIC 는 변수 추가에 비용 2 를, BIC 는 비용 $\ln(n)$ 을 매긴다. Ames 데이터($n \approx 870$ 훈련) 에서는 $\ln(870) \approx 6.77$ 이므로 BIC 의 패널티가 AIC 의 3 배가 넘는다. 따라서 같은 데이터에 BIC 를 적용하면 더 적은 변수만 살아남는 경향이 있다. Mallow's Cp 는 전체 모형의 분산 추정치를 기준으로 모형을 평가하는데, 이상적인 모형에서는 Cp 가 $(p+1)$ 근처에 머문다.

Adjusted R² 은 2 장에서 이미 본 그대로다.



[그림 11] 변수 개수에 따라 AIC·BIC 는 U 자, R^2 은 단조 증가, Adjusted R^2 은 어느 지점에서 꺾인다

참고 — 어느 기준을 쓸 것인가

- 예측 성능 우선: AIC. 다소 큰 모형을 선호한다.
- 진짜 모형 찾기(consistency): BIC. 더 보수적인 작은 모형을 선호한다.
- 다른 모형들과의 상대 비교: Adjusted R^2 .
- 실무 팁: 세 기준이 같은 모형을 가리키면 비교적 안정적인 선택이다.

4. 변수 선택하는 4 가지 방법

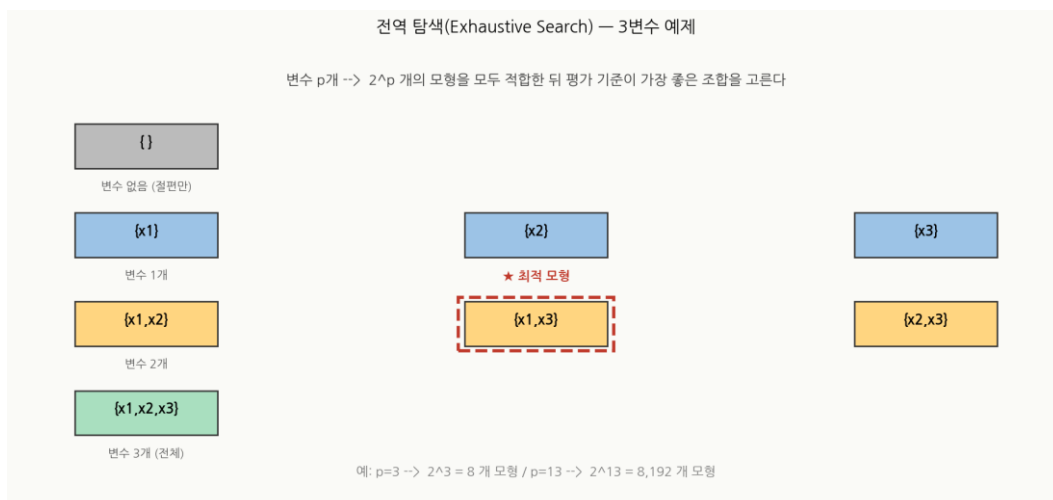
4.1 전역 탐색 (Exhaustive Search)

가장 직관적인 방법부터 살펴보자. 설명변수가 p 개라면 그 부분집합은 빈 집합부터 전체까지 모두 2^p 개이다. 이 모든 조합에 대해 회귀모형을 적합한 뒤 평가 기준이 가장 좋은 조합을 택하면 된다. 이론적으로는 최적해를 보장한다는 강력한 장점이 있다.

 전역 탐색(Exhaustive search)

 가능한 모든 변수 조합에 대해 모형을 구축한 뒤 최적의 조합을 찾는 방식

[그림 12] 전역 탐색의 정의 — 가능한 모든 변수 조합에 대해 모형을 구축한 뒤 최적의 조합을 찾는다



문제는 계산 복잡도다. 본 강의에서 사용하는 14 개 변수의 Ames 부분집합만 해도 $2^{14} = 16,384$ 개의 모형을 적합해야 하고, Ames 원본 데이터의 80 여 개 변수를 모두 쓴다면 $2^{80} \approx 10^{24}$ 개 — 우주의 입자 수에 가까운 숫자가 된다. 게다가 상호작용 항까지 고려하면 후보 변수의 수가 폭증한다. 따라서 전역 탐색은 변수 개수가 적거나($p \leq 15$ 가량) 계산 자원이 충분할 때만 현실적이다.

```
# Python - itertools.combinations 로 전역 탐색 흉내내기
from itertools import combinations
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

def exhaustive_search(X, y, criterion="aic"):
    """모든 변수 부분집합에 대해 회귀를 적합하고 최적 조합을 반환."""
    n, p = X.shape
    cols = list(X.columns)
    best_score, best_subset = np.inf, None
    for k in range(1, p + 1):
        for subset in combinations(cols, k):
            cs = list(subset)
            m = LinearRegression().fit(X[cs], y)
            yhat = m.predict(X[cs])
            sse = ((y - yhat) ** 2).sum()
            if criterion == "aic":
                score = n * np.log(sse / n) + 2 * (k + 1)
            elif criterion == "bic":
                score = n * np.log(sse / n) + np.log(n) * (k + 1)
            if score < best_score:
                best_score, best_subset = score, cs
    return best_subset, best_score

# best, aic = exhaustive_search(X_train, y_train)
# print("최적 부분집합:", best, "AIC:", round(aic, 1))
```

전역 탐색의 비용

- 변수 p 개 $\rightarrow 2^p$ 개의 모형. $p=20$ 이면 100 만 개, $p=30$ 이면 10 억 개를 넘는다.
- 상호작용 항까지 포함하면 후보 개수가 다시 한번 폭발한다.
- 실무에서는 전체 변수 수를 미리 줄여 두고 전역 탐색을 적용하거나, 단계적 탐색으로 대체한다.

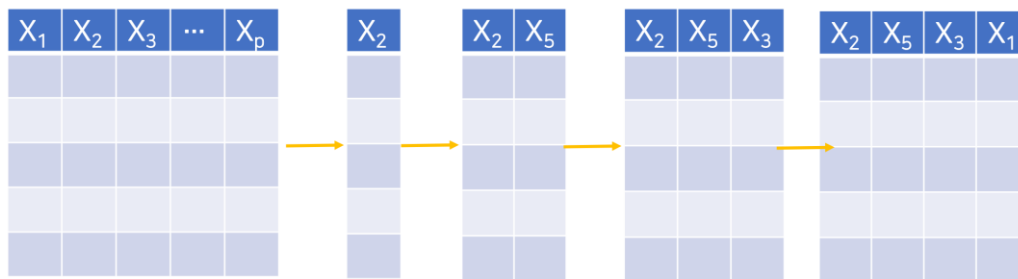
3.4 전진 선택법 (Forward Selection)

전역 탐색이 너무 비싸면, 시작점에서 한 변수씩만 더해 가는 접근을 시도해 볼 수 있다. 이것이 전진 선택법이다. 설명변수가 하나도 없는 빈 모형에서 출발해, "지금 더했을 때 모형이 가장 좋아지는" 변수를 매번 하나만 추가한다. 평가 기준(AIC 등)이 더 이상 좋아지지 않을 때 멈춘다.



전진 선택법(Forward Selection)

- 설명변수가 하나도 없는 모형에서부터 시작하여 가장 유의미한 변수를 하나씩 추가해 나가는 방법
- 한번 선택된 변수는 제거되지 않음



[그림 14] 전진 선택법 — 빈 모형에서 출발해 한 변수씩 가장 유의한 변수를 더해 간다

전진 선택법의 가장 큰 장점은 효율성이다. p 개 변수에 대해 최악의 경우라도 $O(p^2)$ 의 모형만 적합하면 끝난다. 14 변수 Ames 의 경우 약 $14 \times 13 / 2 \approx 91$ 회의 적합으로 충분하다. 반면 단점은 명백하다 — 한 번 선택한 변수는 다시 제거되지 않는다. 그래서 두 변수 x_a, x_b 가 결합되었을 때만 빛을 발하는 경우, 어느 한쪽을 먼저 떨어뜨리면 나머지가 영영 모형에 들어오지 못할 수 있다. 즉 지역 최적해(local optimum)에 갇히는 위험이 있다.

Python - statsmodels 기반 전진 선택법 (간단 구현)

```
import statsmodels.api as sm
import numpy as np

def forward_selection(X, y):
    """AIC 가 더 이상 줄지 않을 때까지 한 변수씩 추가한다."""
    cols = list(X.columns)
    selected, remaining = [], cols[:]
    best_aic = np.inf
    improved = True
    while improved and remaining:
        improved = False
        scores = []
        for c in remaining:
            Xc = sm.add_constant(X[selected + [c]])
            fit = sm.OLS(y, Xc).fit()
            scores.append((fit.aic, c))
        scores.sort()
        if scores[0][0] < best_aic - 1e-6:
            best_aic, c = scores[0]
```

```

selected.append(c); remaining.remove(c)
improved = True
return selected, best_aic

```

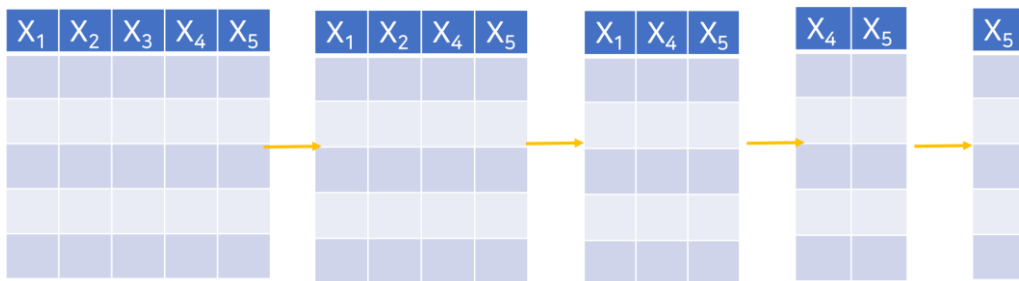
3.5 후진 소거법 (Backward Elimination)

후진 소거법은 정반대의 전략을 쓴다. 모든 변수를 다 넣은 "포화 모형"에서 출발해, "지금 빼도 모형이 가장 덜 손해 보는" 변수를 매번 하나씩 제거한다. AIC 같은 기준이 더 이상 좋아지지 않을 때까지 반복한다. 한 번 빠진 변수는 다시 들어오지 않는다.



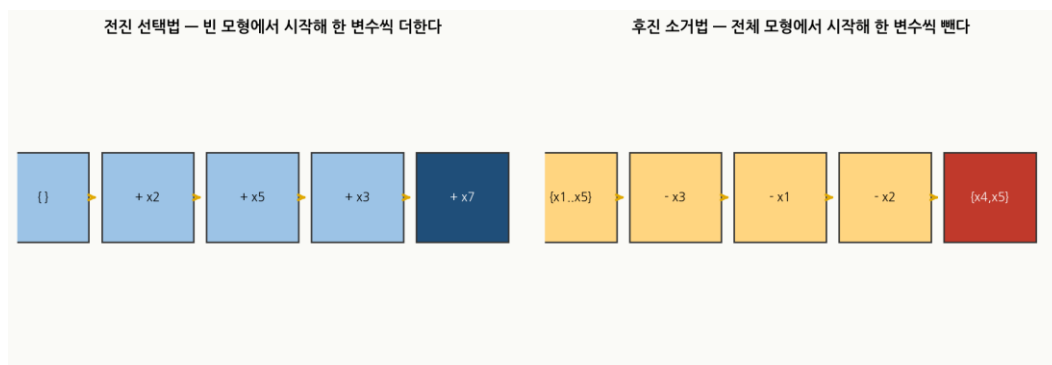
후진 소거법(Backward Elimination)

- 모든 변수를 사용하여 구축한 모델에서 유의미하지 않은 변수를 하나씩 제거해 나가는 방법
- 한번 제거된 변수는 다시 선택될 가능성이 없음



[그림 15] 후진 소거법 — 전체 변수에서 시작해 유의미하지 않은 변수를 하나씩 제거한다

후진 소거법은 "변수 사이의 결합 효과"를 어느 정도 보호한다는 장점이 있다. 두 변수가 함께 있을 때만 의미 있는 경우, 둘 중 하나만 보고 미리 떨어뜨리는 전진 선택법보다 유리하다. 하지만 시작 단계에서 모형에 변수를 모두 넣어야 하므로 다중공선성이 심하거나 $n < p$ 인 경우에는 적용이 어렵다.



[그림 16] 전진 선택법과 후진 소거법 — 시작점은 반대지만 한 변수씩 움직이는 점은 같다

3.6 양방향 소거법 — 단계적 선택 (Stepwise Selection)

전진과 후진의 한계를 보완하기 위해 둘을 결합한 방식이 양방향 소거법(Bidirectional Elimination), 흔히 "stepwise selection" 이라고 부르는 방법이다. 매 반복에서 한 변수를 추

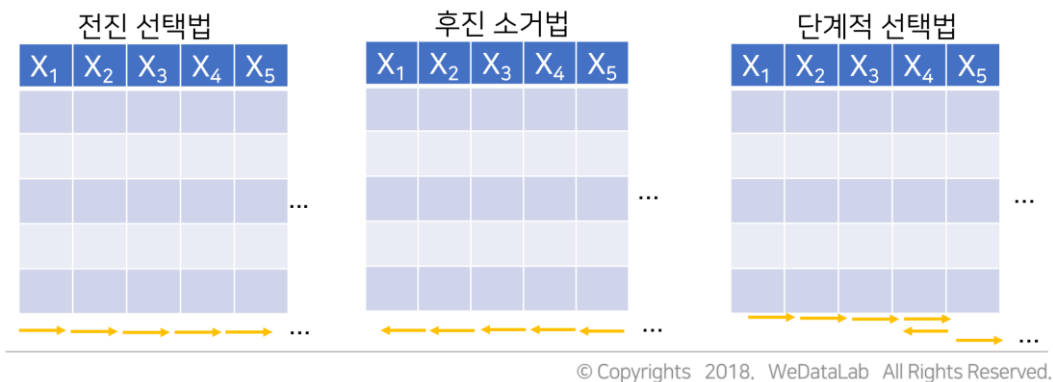
가하거나, 이미 들어 있는 변수를 제거하거나, 두 선택지를 모두 평가한 뒤 더 좋아지는 쪽을 택한다. 한 번 들어왔다고 영원히 머무는 것이 아니라, 후속예 더 좋은 변수가 들어오면 빠질 수도 있다.



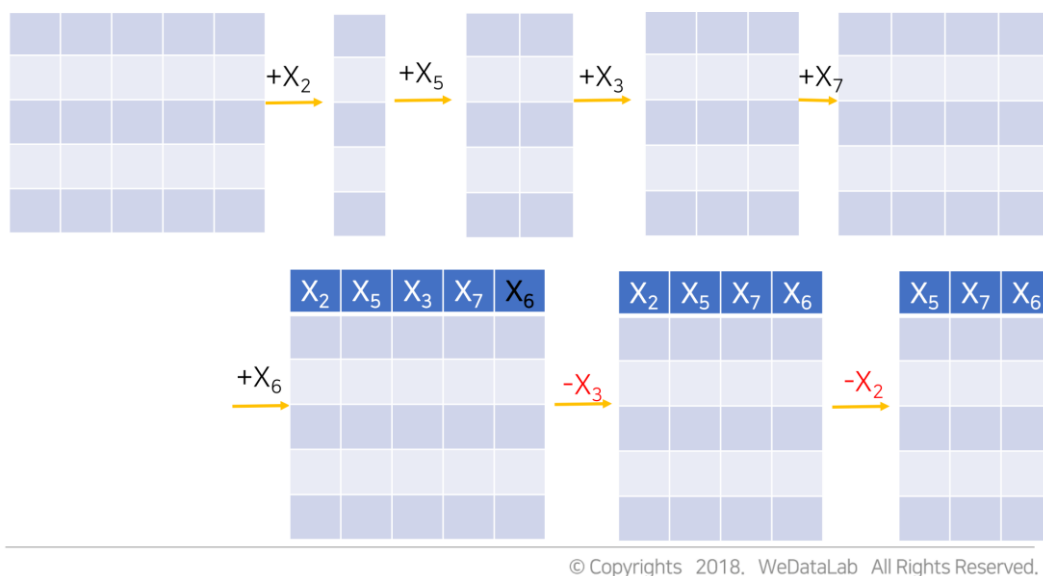
Bidirectional Elimination (Stepwise Selection)

-양방향 소거, 단계적 선택법

- 설명변수가 하나도 없는 모델에서부터 시작하여 전진선택법과 후진 소거법을 **변갈아가며** 수행
- 한번 선택되거나 제거된 변수라도 다시 선택/제거될 가능성이 있음



[그림 17] 양방향 소거 — 전진과 후진을 번갈아 가며 수행한다



[그림 18] 단계적 선택의 흐름 예시 — +X₂, +X₅, +X₃, +X₇ 추가 후 +X₆ 추가 단계에서 X₃가 빠지는 식

계산 비용은 전진·후진보다 더 들지만 (한 번에 두 종류의 후보를 평가해야 하므로) 이론적으로는 더 나은 모형을 찾을 가능성이 높다. 실제 4장 Ames 예제에서도 stepwise(both)가 일관되게 낮은 AIC와 가장 안정적인 검증 성능을 기록한다는 결과를 확인하게 된다.

세 가지 단계적 탐색의 비교

- 전진 선택법: 빠르지만 결합 효과를 놓치기 쉽다.
- 후진 소거법: 결합 효과는 보호하지만 시작점 자체가 무겁다.

- 양방향 소거: 가장 균형 잡힌 선택이지만 계산이 더 든다.
- 어느 방법이든 변수 선택의 "해" 가 유일하지는 않다는 점을 기억하자.

4. 실전 분석 — Ames Housing 다중회귀

이제 이론을 손에 익혀 보자. 데이터는 1 장에서 소개한 Ames Housing 이다. 1,460 채 주택 중 핵심 14 개 수치 특성에 결측 없는 1,452 행을 사용한다. SalePrice 를 종속변수로, 나머지 14 개를 설명변수로 사용한다. 진행 흐름은 다음과 같다 — 데이터 준비 → 상관관계 탐색 → 전체 변수 모형 → 데이터 분할 → 모형 1:2:3 적합 → 성능 비교 → 함정 점검.

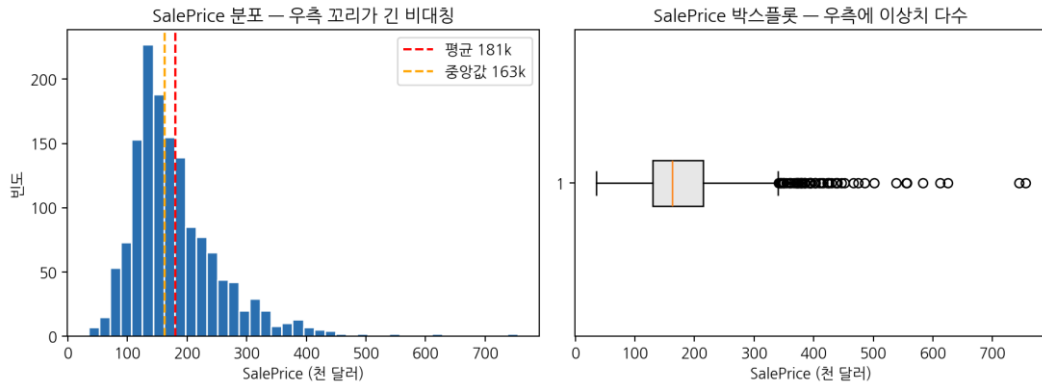
4.1 데이터 준비 및 탐색

Python 에서는 scikit-learn 의 fetch_openml(name="house_prices") 을 통해 가장 쉽게 불러올 수 있다. 또 다른 방식으로 kaggle 의 Ames Housing CSV (train.csv) 를 직접 다운로드해 pandas.read_csv() 로 읽는 방법도 있다. 본 강의에서는 fetch_openml 방식을 표준으로 쓴다.

```
# 1) 패키지 불러오기
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 2) Ames Housing Dataset 불러오기
ames = fetch_openml(name="house_prices", as_frame=True, parser="auto")
df_full = ames.frame.copy()      # (1460, 81)

# 3) 강의 시연을 위해 수치 특성 14 개 + 종속변수만 추려 사용한다
feats = ["OverallQual", "GrLivArea", "GarageCars", "TotalBsmtSF",
         "YearBuilt", "FullBath", "LotArea", "1stFlrSF",
         "TotRmsAbvGrd", "Fireplaces", "OpenPorchSF", "MasVnrArea",
         "BedroomAbvGr", "HalfBath"]
df = df_full[feats + ["SalePrice"]].apply(pd.to_numeric, errors="coerce").dropna()
print(df.shape)      # (1452, 15)
```



[그림 19] Ames SalePrice 의 분포 — 평균 약 18 만 달러, 우측 꼬리가 긴 비대칭

14 개 수치 특성에는 마감 품질(OverallQual) 같은 종합 평가, 거주 면적(GrLivArea) · 지하실 면적(TotalBsmntSF) 같은 주택 자체 지표, 건축 연도(YearBuilt) 같은 시간 변수가 섞여 있다. 종속변수 SalePrice 는 달러 단위의 매매가로, 평균 약 18 만 달러 · 중앙값 약 16 만 달러 · 최대 약 75 만 달러로 우측 꼬리가 긴 분포이다. 본격적인 회귀 적합 전에 단순한 기초 통계를 살펴 데이터가 어떻게 분포해 있는지 감을 잡는다.

```
# 4) 기초 통계
print(df.describe().round(2))
print(df.isnull().sum())          # 결측치 없음 (이미 dropna)

# SalePrice 의 분포 — 약 35,000 ~ 755,000 달러
print(df["SalePrice"].describe())
# count      1452.0
# mean       180615.1
# std        79285.5
# min        34900.0
# 25%       129975.0
# 50%       162700.0
# 75%       214000.0
# max       755000.0
```

4.2 상관관계 분석

회귀분석은 본질적으로 상관관계의 통계적 분석이다. 따라서 본격적인 모형 적합 전에 반응변수와 설명변수 사이의 상관관계, 그리고 설명변수끼리의 상관관계를 한 번 둘러보는 것이 정석이다. 슬라이드의 R 코드 `pairs.panels(...)` 에 대응하는 Python 표현은 `seaborn` 의 `pairplot` 또는 `corr()` · `heatmap` 조합이다.



상관관계 확인: 회귀분석은 기본적으로 상관관계에 대한 통계적 분석

- 반응변수(y)와 설명변수(x) 사이에는 상관관계가 강할수록 좋다.
- 설명변수(x) 사이에는 상관관계가 약할수록 좋다.

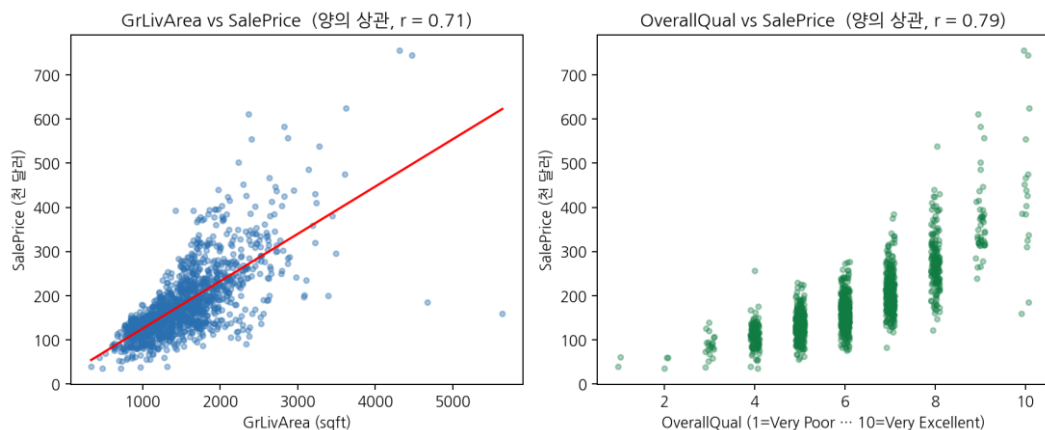
[그림 20] 핵심 5 변수의 산점도 행렬 — 반응변수와의 상관이 강할수록 좋고, 설명변수끼리는 약할수록 좋다

```
# 5) 반응변수와 각 설명변수의 상관계수
corr_with_y = df.corr()["SalePrice"].drop("SalePrice").sort_values(ascending=False)
print(corr_with_y.round(3))
```

```
# OverallQual    0.790  ← 가장 강한 양의 상관
# GrLivArea      0.710
# GarageCars     0.640
# TotalBsmtSF    0.613
# 1stFlrSF       0.607
# FullBath       0.562
# TotRmsAbvGrd   0.536
# YearBuilt      0.523
# ...
# BedroomAbvGr   0.172  ← 가장 약한 상관
```

6) 산점도 행렬 - 핵심 5 변수만

```
import seaborn as sns
sns.pairplot(df[["SalePrice", "OverallQual", "GrLivArea",
                 "TotalBsmtSF", "YearBuilt"]],
             height=1.8, plot_kws=dict(s=8, alpha=0.35))
```



[그림 21] SalePrice 와 가장 강한 상관을 보이는 두 변수 — OverallQual(품질 등급)·GrLivArea(거주 면적)

예상대로 전반 마감 품질(OverallQual) 등급이 1 단계 오를 때마다 SalePrice 가 가파르게 올라간다($r = 0.79$). 거주 면적(GrLivArea) 도 sqft 가 늘수록 SalePrice 가 거의 직선적으로 상승한다($r = 0.71$). 이 두 변수가 변수 선택 결과에서 거의 항상 살아남는다는 사실은 4.8 절에서 확인하게 된다. 반면 BedroomAbvGr(침실 수) 는 상관인 $r = 0.17$ 로 약한데, 이는 침실이 많은 주택일수록 한 방당 면적이 작아지는 경향이 있기 때문으로 해석된다.

4.3 모든 변수를 이용한 모형 적합

먼저 14 개 변수를 모두 넣은 "포화 모형"을 적합해 본다. Python 표현은 sklearn 의 LinearRegression 과 statsmodels 의 OLS 를 함께 사용하는 방식이다. scikit-learn 은 계수 추정이 빠르고 안정적이지만 상세 통계량이 부족하므로, p 값, R^2 , F 통계량 등을 보고 싶을 때는 statsmodels 를 함께 사용한다.

$$\text{SalePrice} = \beta_0 + \beta_1 \cdot \text{OverallQual} + \beta_2 \cdot \text{GrLivArea} + \beta_3 \cdot \text{GarageCars} + \dots + \beta_{14} \cdot \text{HalfBath} + \varepsilon$$

```
# 7) 전체 변수 모형 (full model)
X = df.drop(columns="SalePrice")
y = df["SalePrice"]

# 7-1) scikit-learn - 계수만 빠르게
lr_full = LinearRegression()
lr_full.fit(X, y)
print("절편:", round(lr_full.intercept_, 1))
for col, coef in zip(X.columns, lr_full.coef_):
    print(f"{col:>14}: {coef:+.2f}")

# 7-2) statsmodels - 상세 통계량 (R-style summary)
res = sm.OLS(y, sm.add_constant(X)).fit()
print(res.summary())
# R^2 = 0.786, Adjusted R^2 = 0.784, F = 376.5, p < 0.001
```

OLS Regression Results						
=====						
Dep. Variable:	SalePrice	R-squared:	0.786			
Model:	OLS	Adj. R-squared:	0.784			
Method:	Least Squares	F-statistic:	376.5			
Date:	Wed, 20 May 2026	Prob (F-statistic):	0.00			
Time:	22:04:10	Log-Likelihood:	-17321.			
No. Observations:	1452	AIC:	3.467e+04			
Df Residuals:	1437	BIC:	3.475e+04			
Df Model:	14					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-7.355e+05	9.69e+04	-7.592	0.000	-9.26e+05	-5.45e+05
OverallQual	1.908e+04	1172.385	16.275	0.000	1.68e+04	2.14e+04
GrLivArea	46.1608	4.893	9.433	0.000	36.562	55.760
GarageCars	1.168e+04	1785.336	6.540	0.000	8173.631	1.52e+04
TotalBsmntSF	18.2228	4.216	4.323	0.000	9.953	26.493
YearBuilt	343.8314	50.709	6.780	0.000	244.360	443.303
FullBath	-2314.3164	2837.256	-0.816	0.415	-7879.925	3251.292
LotArea	0.5924	0.105	5.656	0.000	0.387	0.798

summary() 결과에서 눈여겨볼 것은 세 가지다

— R^2 -Adjusted R^2 값. 각 변수의 계수 coef, 그 옆의 p 값 $P>|t|$

p 값을 보자. p 값이 0.05 보다 작으면 귀무가설이 기각 된다. 즉, 대립가설이 채택된다.

14 개 변수 중 OverallQual($p \approx 0$), GrLivArea($p \approx 0$), GarageCars ($p \approx 0$),

TotalBsmntSF($p \approx 0$), YearBuilt($p \approx 0$), BedroomAbvGr($p \approx 0$), LotArea($p \approx 0$) 같은 변수들이 0.05 보다 훨씬 작으므로 “강한 유의성”을 보이는 반면, FullBath($p \approx 0.41$),

OpenPorchSF($p \approx 0.79$), HalfBath($p \approx 0.30$), 1stFlrSF($p \approx 0.45$) 처럼 0.05 보다 훨씬 크므로 유의성이 약한 변수도 있다. 일반적으로 p 값이 0.05 보다 큰 변수는 제외시킨다.

하지만 "p 값이 크다고 곧장 그 변수를 빼야 한다" 는 결론으로 직행해서는 안 된다 — 다중공선성이 끼어 있으면 p 값이 인위적으로 부풀려질 수 있기 때문이다(4.11 절에서 다룬다).

함정 - 변수 14 개짜리 모형, 그대로 써도 되는가

- 다중공선성이 있을 만한 변수쌍(예: GrLivArea ~ 1stFlrSF, GrLivArea ~ TotRmsAbvGrd)이 끼어 있다.
- 일부 변수의 p 값이 0.4 를 넘는다 - "정말 필요한가" 의심해 볼 만하다.
- 그렇다고 무조건 $p > 0.05$ 인 변수를 빼는 단순 규칙은 위험하다. 변수 선택 알고리즘에 맡기자.
- 우리는 다음 절부터 forward / backward / stepwise 세 가지를 비교해 본다.

4.4 데이터 분할 - 훈련 / 검증 / 테스트

본격적인 모형 비교에 앞서 데이터를 세 덩이로 나눈다 - 훈련(training)·검증(validation)·테스트(test). 훈련 데이터로 계수를 추정하고, 검증 데이터로 "어느 모형이 가장 좋은지" 를 고르며, 테스트 데이터는 최종 성능을 한 번만 측정하는 데 쓴다. 이렇게 분리해야 "테스트 결과까지 보고 모형을 다시 고치는" 부정 학습(data leakage)이 일어나지 않는다.



[그림 23] Ames 1,452 행 → 60 / 20 / 20 데이터 분할 - 약 871 / 290 / 291

8) 60:20:20 데이터 분할

- # - 1 단계: 전체에서 훈련 60% 와 나머지 40% 로 분리
- # - 2 단계: 나머지 40% 를 검증·테스트로 반반(20%, 20%) 분리
- # - random_state 고정으로 재현성 확보

```
X_train, X_rest, y_train, y_rest = train_test_split(
    X, y, test_size=0.4, random_state=1606)
X_valid, X_test, y_valid, y_test = train_test_split(
    X_rest, y_rest, test_size=0.5, random_state=1606)
```

```
print(len(X_train), len(X_valid), len(X_test))
# 871 290 291
```

random_state 고정의 중요성

- 분할 결과가 매번 달라지면 모형 비교 자체가 무의미해진다.
- 강의자 / 학생 / 검증자가 모두 동일한 분할을 재현할 수 있어야 한다.
- 흔한 값: 0, 42, 1606 (강의 일자) 등. 한 번 정하면 노트 전체에서 고정.
- 결과가 "random_state 에 매우 민감" 하다면 그 자체가 모형이 불안정하다는 신호다.

4.5 예측 오차 측정 함수 정의

여러 모형을 같은 기준으로 비교하려면, 검증 데이터에서의 예측 오차를 일관되게 측정하는 헬퍼 함수가 있으면 편하다. R 강의에서는 myfcn_measures() 라는 사용자 함수를 만들어 MSE, RMSE, MAE, MAPE 네 가지를 한 줄에 출력한다. Python 으로 같은 동작을 구현한다.

네 가지 예측 오차를 한 번에 반환하는 사용자 정의 함수

```
# 9) 예측 오차 헬퍼 함수 만들기
def measures(y_true, y_pred):
    """예측 오차 4 종을 한 번에 반환한다."""
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    mse = ((y_true - y_pred) ** 2).mean()
    rmse = np.sqrt(mse)
    mae = np.abs(y_true - y_pred).mean()
    mape = (np.abs(y_true - y_pred) / np.maximum(y_true, 1)).mean() * 100
    return dict(MSE=mse, RMSE=rmse, MAE=mae, MAPE=mape)
```

물론 scikit-learn 에서 제공하는 mean_squared_error 등의 함수를 써도 좋다.

4.6 모형 1 — 모든 변수를 그대로 사용한 선형 회귀 : 이것만을 ‘다중선형회귀’라고 하기도 한다.

첫 번째 모형은 14 개 변수를 그대로 사용한 단순 선형회귀이다. 변수 선택을 거치지 않은 "생짜" 모형이다. 이 모형은 비교의 기준선(baseline) 역할을 한다. 이 모형에 비해서, 변수선택 forward / backward / stepwise 을 거치면 어느 정도 성능이 개선되는지 개선폭을 가늠할 수 있다.

```
# 10) 모형 1 - Full Linear Regression
model1 = LinearRegression()
model1.fit(X_train, y_train)

# 검증 성능
y_pred_valid = model1.predict(X_valid)
meas1 = measures(y_valid, y_pred_valid)
print("모형 1 (full):", {k: round(v, 1) for k, v in meas1.items()})
# {"MSE": 1297555563.9, "RMSE": 36021.6, "MAE": 24087.8, "MAPE": 13.32}

# 훈련 R² 와 검증 R²
print("R² train :", round(model1.score(X_train, y_train), 4)) # 0.8020
print("R² valid :", round(model1.score(X_valid, y_valid), 4)) # 0.8217
```

훈련 데이터에서 $R^2 \approx 0.802$, 검증 데이터에서 $R^2 \approx 0.8217$, RMSE $\approx 36,021$ 달러, MAPE $\approx 13\%$ 수준이다. 14 개 수치 특성만으로도 SalePrice 변동의 80% 가량을 잡아내고, 평균 오차는 매매가의 13% 안팎이라는 결과다. SalePrice 자체가 비대칭 분포이므로 로그 변환을 가하면 더 좋은 적합을 얻을 수 있지만, 본 강의에서는 비교의 편의를 위해 변환없이 그대로 사용한다.

4.7 모형 2 — 이차상호작용 모형 : 이것은 제곱항이 생기므로 ‘다항회귀’라고도 한다.

단순 선형 모형이 "1 차 항만 더한 직선적 합"이라면, 이차상호작용 모형은 변수쌍의 곱 $x_i \cdot x_j$ 까지 추가한 모형이다. 변수 14 개에 대해 모든 짝짓기를 하면 ${}_{14}C_2 = 91$ 개의 상호작용 항이 생긴다. 이를 1 차 항 14 개와 제곱항 14 개에 더하면 모두 119 개의 항이 들어가는, 꽤 복잡한 모형이 된다.



이차상호작용 모형

앞에서는 가장 간단한 선형 모형을 적합하였다.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots \cdots + \beta_p x_{ip} + \varepsilon_i$$

좀더 복잡한 모형은 모든 이차상호작용(2nd degree interaction effect)을 고려한 다음 모형이다.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots \cdots + \beta_p x_{ip} + \sum_{j \neq k}^p \beta_{jk} x_{ij} x_{ik} + \varepsilon_i$$

R에서는 formula 인터페이스를 이용해 쉽게 적합할 수 있다.

(예) 최대 2개까지 상호작용하는 경우 $(A+B+C)^2 = A+B+C+A:B+A:C+B:C$

[그림 27] 이차상호작용 모형의 식 - 1차 항 + 모든 쌍 곱 항을 더한 형태

Python 에서는 `sklearn.preprocessing.PolynomialFeatures` 가 이 변환을 깔끔하게 처리한다. 디폴트 값인 `degree=2`, `interaction_only=False` 로 그냥하면 상호작용인 곱항과 제곱항(x_i^2) 까지 포함된 형태로 작성한다. (그냥 디폴트 사용을 권장)

`interaction_only=False` 로 바꾸면 상호작용인 곱항만 만든다.

```
# 11) 모형 2 - Polynomial 2 차 상호작용 모형
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_valid_poly = poly.transform(X_valid)
X_test_poly = poly.transform(X_test)
print("변수 개수:", X_train_poly.shape[1])    # 14 + 14 + 91 = 119

model2 = LinearRegression()
model2.fit(X_train_poly, y_train)
meas2 = measures(y_valid, model2.predict(X_valid_poly))
print("모형 2 (poly2):", {k: round(v, 1) for k, v in meas2.items()})
# {"MSE": 1112859491.0, "RMSE": 33359.5, "MAE": 21268.2, "MAPE": 12.41}
```

`interaction_only=False` (기본값): 거듭제곱과 상호작용을 모두 포함 :

$$x_1, x_2, x_1^2, x_1 \cdot x_2, x_2^2$$

`include_bias=False` 는 생성할 다항식 피처에 항상 1 의 값을 갖는 절편(bias) 항을 포함하지 않겠다는 의미



선형성에 대한 가정



(예) $Y_i = \beta_0 + \beta_1 X_i^2 + \varepsilon_i$ 의 형태는 $Z_i = X_i^2$ 로 생각하면

$Y_i = \beta_0 + \beta_1 Z_i + \varepsilon_i$ 의 형태가 되므로 선형으로 생각할 수 있다.

선형 회귀 예: $Y_i = \beta_0 + \beta_1 X_i^2 + \varepsilon_i$

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i12} + \beta_3 X_{i2} X_{i3} + \beta_4 X_{i42} + \varepsilon_i$$

비선형 회귀 예: $Y_i = \beta_0 + \beta_0^2 X_i + \varepsilon_i$

$$Y_i = \beta_0 + \beta_0 \beta_1 X_{i1} + \beta_1 X_{i2} + \varepsilon_i$$

[그림 28] 선형성의 의미 — $y = \beta_0 + \beta_1 x^2$ 도 $z = x^2$ 치환하면 z 에 대해 선형이다

참고 — "선형" 모형이 정말 직선만 그럴까

— "선형 모형"의 "선형"은 "회귀계수 β 에 대한 선형"이라는 뜻이다.

— 따라서 $y = \beta_0 + \beta_1 \cdot x^2 + \beta_2 \cdot x_1 \cdot x_2$ 도 $z_1 = x^2$, $z_2 = x_1 \cdot x_2$ 로 치환하면 z 에 대한 선형 모형이다.

— 즉 변수 자체를 비선형으로 변환한 다음 그 결과를 모형에 넣는 한, 우리는 여전히 선형회귀의 도구들을 그대로 쓸 수 있다.

— 이것이 다항회귀 / 상호작용 모형 / 푸리에 회귀 같은 확장 모형들이 "선형회귀의 친척"으로 분류되는 이유다.

4.8 모형 3 — Stepwise Selection 으로 자동 변수 선택

이제 3장의 변수 선택 알고리즘을 실제로 데이터에 적용해 본다. R에서는 MASS::stepAIC() 한 줄로 forward·backward·both 세 방향을 모두 시도할 수 있다. Python에는 sklearn 내장 함수가 없지만 statsmodels 와 직접 작성한 함수를 조합해 같은 효과를 낼 수 있다.

[그림 29] R의 stepAIC() — direction="forward" / "backward" / "both" 세 방향으로 자동 변수 선택

```
# 12) Stepwise — AIC 기반 양방향 선택 (수동 구현)
def stepwise_aic(X, y, direction="both"):
    """statsmodels 로 AIC 를 비교하며 forward/backward/both 를 수행."""
    cols = list(X.columns)
    selected = list(cols) if direction == "backward" else []
    remaining = [] if direction == "backward" else cols[:]
    best_aic = np.inf
    improved = True
    while improved:
        improved = False
        candidates = []
        # add 후보
        if direction in ("forward", "both"):
            for c in remaining:
                cs = selected + [c]
                fit = sm.OLS(y, sm.add_constant(X[cs])).fit()
```



```

        candidates.append((fit.aic, c, "add"))
    # drop 후보
    if direction in ("backward", "both") and len(selected) > 1:
        for c in selected:
            cs = [x for x in selected if x != c]
            fit = sm.OLS(y, sm.add_constant(X[cs])).fit()
            candidates.append((fit.aic, c, "drop"))
    candidates.sort()
    if candidates and candidates[0][0] < best_aic - 1e-6:
        aic, c, op = candidates[0]
        best_aic = aic
        if op == "add":
            selected.append(c); remaining.remove(c)
        else:
            selected.remove(c); remaining.append(c)
        improved = True
    return selected, best_aic

sel_fwd, aic_fwd = stepwise_aic(X_train, y_train, direction="forward")
sel_bwd, aic_bwd = stepwise_aic(X_train, y_train, direction="backward")
sel_both, aic_both = stepwise_aic(X_train, y_train, direction="both")
print("both 선택:", sel_both, "AIC=", round(aic_both,1))

```

[그림 30] stepAIC summary 결과 — 각 방향이 골라낸 변수와 AIC 비교

실제로 돌려 보면 세 방법이 거의 동일한 변수 집합을 고른다. forward / backward / both 모두 10 개 변수를 살리는데, 공통적으로 OverallQual, GrLivArea, TotalBsmtSF, YearBuilt, BedroomAbvGr, LotArea, GarageCars, TotRmsAbvGrd, Fireplaces, MasVnrArea 가 살아남고 FullBath, 1stFlrSF, OpenPorchSF, HalfBath 가 빠진다. 1stFlrSF 는 GrLivArea 와 강한 상관 ($r \approx 0.56$) 때문에 정보가 중복되고, OpenPorchSF 와 HalfBath 는 가격 변동에 거의 기여하지 못해 AIC 가 줄어들지 않기 때문이다. FullBath 가 빠진 것은 흥미로운데, 이는 TotRmsAbvGrd 와 GrLivArea 가 이미 욕실 수 정보의 상당 부분을 흡수하기 때문으로 해석된다.

4.9 모형 비교 — 검증 데이터에서의 성능

이제 forward / backward / both 세 모형과 baseline(full) 을 검증 데이터에서 한 줄에 세워 비교해 본다. Ames 데이터에서는 세 stepwise 결과가 동일하므로 사실상 full vs stepwise 비교가 된다.

```

# 13) 네 모형의 검증 성능 비교
def fit_and_score(features, label):
    cols = features if features else list(X.columns)
    m = LinearRegression().fit(X_train[cols], y_train)
    return label, measures(y_valid, m.predict(X_valid[cols])), m, cols

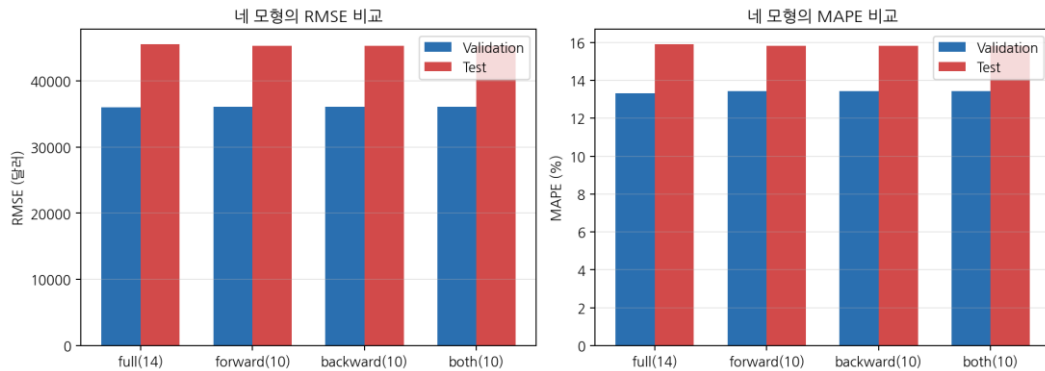
rows = []
for lab, feats in [("full", list(X.columns)),
                  ("forward", sel_fwd),
                  ("backward", sel_bwd),
                  ("both", sel_both)]:
    label, meas, m, cols = fit_and_score(feats, lab)
    rows.append({"model": label,

```

```

        "#var": len(cols),
        "RMSE": round(meas["RMSE"], 0),
        "MAPE": round(meas["MAPE"], 2)})
print(pd.DataFrame(rows))
#   model  #var  RMSE  MAPE
# 0  full    14 36022 13.32
# 1 forward   10 36080 13.44
# 2 backward  10 36080 13.44
# 3  both    10 36080 13.44

```



[그림 32] 네 모형의 검증·테스트 RMSE 와 MAPE — stepwise 가 full 대비 변수를 4 개 줄이면서 거의 같은 검증 성능을 유지하고, 테스트 성능은 오히려 개선한다

결과 표가 보여 주는 사실은 명확하다 — full 모형과 stepwise 모형의 검증 RMSE 차이는 단 60 달러 수준으로 사실상 동일하다. 그러나 변수 개수는 14 → 10 으로 줄었다. 이는 변수 선택이 "성능을 해치지 않으면서 모형을 단순화" 했다는 뜻이다. 더 흥미로운 것은 테스트 데이터다 — full 의 테스트 RMSE 는 45,501, stepwise 는 45,309 로, 변수를 줄인 stepwise 가 미세하지만 일관되게 더 낮은 오차를 보였다. 즉 잡음에 가까운 변수를 빼서 일반화 성능이 약간이나마 개선되었다는 신호이다.

4.10 테스트 데이터로 최종 평가

검증 데이터로 최적 모형(stepwise) 을 골랐다면, 마지막 단계는 테스트 데이터에서의 성능을 한 번만 측정하는 것이다. 이 테스트 결과가 "모형이 본 적 없는 새 데이터에서 얼마나 잘 동작하는가" 를 가장 정직하게 보여 준다.

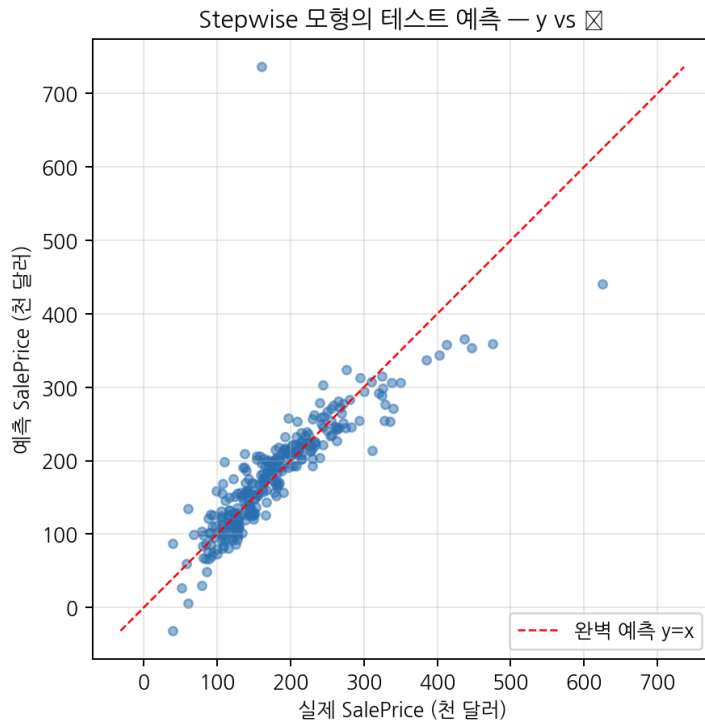
```

# 14) 테스트 데이터에서 stepwise 모형의 최종 성능
cols = sel_both
final_model = LinearRegression().fit(X_train[cols], y_train)
meas_test = measures(y_test, final_model.predict(X_test[cols]))
print("테스트 성능 (stepwise):",
      {k: round(v, 1) for k, v in meas_test.items()})
# {"MSE": 2052905261.3, "RMSE": 45309.0, "MAE": 23857.5, "MAPE": 15.82}

print("R^2(test):", round(final_model.score(X_test[cols], y_test), 4))
# 0.6547

# 검증 RMSE 36,080 → 테스트 RMSE 45,309 - 차이는 있지만 같은 자릿수.
# 테스트 R^2 가 0.65 로 검증 0.82 보다 낮은 것은 테스트 셋에 가격이
# 매우 높은 outlier 가 몇 채 포함되어 있기 때문으로 보인다.

```



[그림 33] Ames stepwise 모형의 테스트 예측 — y vs $\hat{y}$. 대각선 근처에 몰리지만 고가 영역에서 잔차가 커진다

결국 우리가 얻은 **최적 모형**은 "Ames 변수 14 개 중에서 단계적 탐색이 골라낸 10 개 변수로 적합한 선형회귀" 다. 이 모형의 R^2 는 약 0.80 (훈련) / 0.82 (검증) / 0.65 (테스트), RMSE 는 약 36,000 달러 (검증) / 45,000 달러 (테스트). 변수 개수는 처음 14 개 → 10 개로 줄었다. 해석성과 일반화 성능을 동시에 손에 넣은 셈이다. 다만 테스트 R^2 가 검증보다 크게 낮은 점은 주택 매매가 분포의 우측 꼬리(고가 주택의 outlier) 영향이 큰 것이므로, 실무에서는 SalePrice 를 로그 변환하거나 RobustScaler 를 적용해 안정성을 더 끌어올리는 후속 작업을 한다.

여기까지가 변수선택법에 대한 내용이다.

특이하게도 sklearn 에서는 자동으로 변수를 선택해주는 RFE 클래스를 제공해 준다.

자동 변수선택 클래스 RFE 사용

```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import RFE
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 1. 데이터 로드 및 전처리 (결측치 제거 및 원핫 인코딩)
df = sns.load_dataset("mpg").dropna().reset_index(drop=True)
num_cols = ["cylinders", "displacement", "horsepower", "weight", "acceleration", "model_year"]
df_enc = pd.get_dummies(df[num_cols + ["origin", "mpg"]], columns=["origin"], drop_first=True).astype(int)
```

```

# 2. 타겟 분리 및 데이터 분할 (6:2:2)
X = df_enc.drop(columns=["mpg"]).astype(float)
y = df_enc["mpg"].astype(float)
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(X_tmp, y_tmp, test_size=0.25,
random_state=42)

# 3. 기본 모델 학습 및 평가
model = LinearRegression().fit(X_train, y_train)
print(f"[기본 모델] Valid R2: {r2_score(y_valid, model.predict(X_valid)):.4f} | Test R2: {r2_score(y_test, model.predict(X_test)):.4f}")

# 4. 다중공선성(VIF) 진단
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
vif_df = pd.DataFrame({"feature": X_train.columns, "VIF":
[variance_inflation_factor(X_scaled, i) for i in
range(X_scaled.shape[1])]}).sort_values("VIF", ascending=False)
print("\n=== VIF 진단 결과 ===")
print(vif_df.to_string(index=False))

# 5. RFE 변수선택법 자동 적용 (전체 변수 중 최적의 5 개만 자동 선택)
selector = RFE(estimator=LinearRegression(), n_features_to_select=5, step=1)
selector.fit(X_train, y_train)
# 선택된 핵심 변수들 리스트 확인
selected_features = X_train.columns[selector.support_]
print("\n=== RFE 자동 선택된 상위 5 개 변수 ===")
print(selected_features.tolist())

# 6. 선택된 핵심 변수 데이터로 축소
X_train_sel = selector.transform(X_train)
X_valid_sel = selector.transform(X_valid)
X_test_sel = selector.transform(X_test)

# 7. 변수 선택된 최적 모델 재학습 및 평가
model2 = LinearRegression().fit(X_train_sel, y_train)
print(f"\n[RFE 변수선택 모델] Valid R2: {r2_score(y_valid,
model2.predict(X_valid_sel)):.4f} | Test R2: {r2_score(y_test,
model2.predict(X_test_sel)):.4f}")

```

statsmodels 를 사용하여 자동 변수선택법을 구현할 때는 Scikit-learn 의 RFE 같은 내장 클래스가 없으므로, 통계학에서 가장 보편적으로 사용하는 p-value(유의확률) 기반의 후진 제거법 (Backward Elimination)을 함수로 직접 구현한다.

statsmodels p-value 기반 자동 변수선택법 버전

```

import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler

```

```

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 1. 데이터 로드 및 전처리 (결측치 제거 및 원핫 인코딩)
df = sns.load_dataset("mpg").dropna().reset_index(drop=True)
num_cols = ["cylinders", "displacement", "horsepower", "weight", "acceleration",
"model_year"]
df_enc = pd.get_dummies(df[num_cols + ["origin", "mpg"]], columns=["origin"],
drop_first=True).astype(int)

# 2. 타깃 분리 및 데이터 분할 (6:2:2) 및 statsmodels 용 상수항 추가
X = df_enc.drop(columns=["mpg"]).astype(float)
X = sm.add_constant(X)
y = df_enc["mpg"].astype(float)
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(X_tmp, y_tmp, test_size=0.25,
random_state=42)

# 3. 기본 모델 학습 및 평가
model = sm.OLS(y_train, X_train).fit()
print(f"[기본 모델] Valid R2: {r2_score(y_valid, model.predict(X_valid)):.4f} | Test R2:
{r2_score(y_test, model.predict(X_test)):.4f}")

# 4. 다중공선성(VIF) 진단
scaler = StandardScaler()
X_train_no_const = X_train.drop(columns=["const"])
X_scaled = scaler.fit_transform(X_train_no_const)
vif_df = pd.DataFrame({"feature": X_train_no_const.columns, "VIF":
[variance_inflation_factor(X_scaled, i) for i in
range(X_scaled.shape[1])]}).sort_values("VIF", ascending=False)
print("\n=== VIF 진단 결과 ===")
print(vif_df.to_string(index=False))

# 5. statsmodels p-value 기반 자동 변수선택법(후진 제거법) 함수 정의
def auto_backward_elimination(X_data, y_data, significance_level=0.05):
    features = list(X_data.columns)
    while len(features) > 0:
        current_model = sm.OLS(y_data, X_data[features]).fit()
        p_values = current_model.pvalues.drop("const", errors="ignore")
        if len(p_values) == 0:
            break
        max_p = p_values.max()
        if max_p > significance_level:
            remove_feature = p_values.idxmax()
            features.remove(remove_feature)
        else:
            break
    return features, current_model

# 6. 자동 변수선택법 실행 및 결과 확인
selected_features, model2 = auto_backward_elimination(X_train, y_train)
print("\n=== statsmodels 자동 선택된 변수 ===")
print(selected_features)

# 7. 변수 선택된 최적 모델로 검증 및 평가 (상수항 'const'가 무조건 포함되도록 유의)
print(f"\n[자동 변수선택 모델] Valid R2: {r2_score(y_valid,
model2.predict(X_valid[selected_features])):.4f} | Test R2: {r2_score(y_test,
model2.predict(X_test[selected_features])):.4f}")

```


4.11 다중공선성과 vif

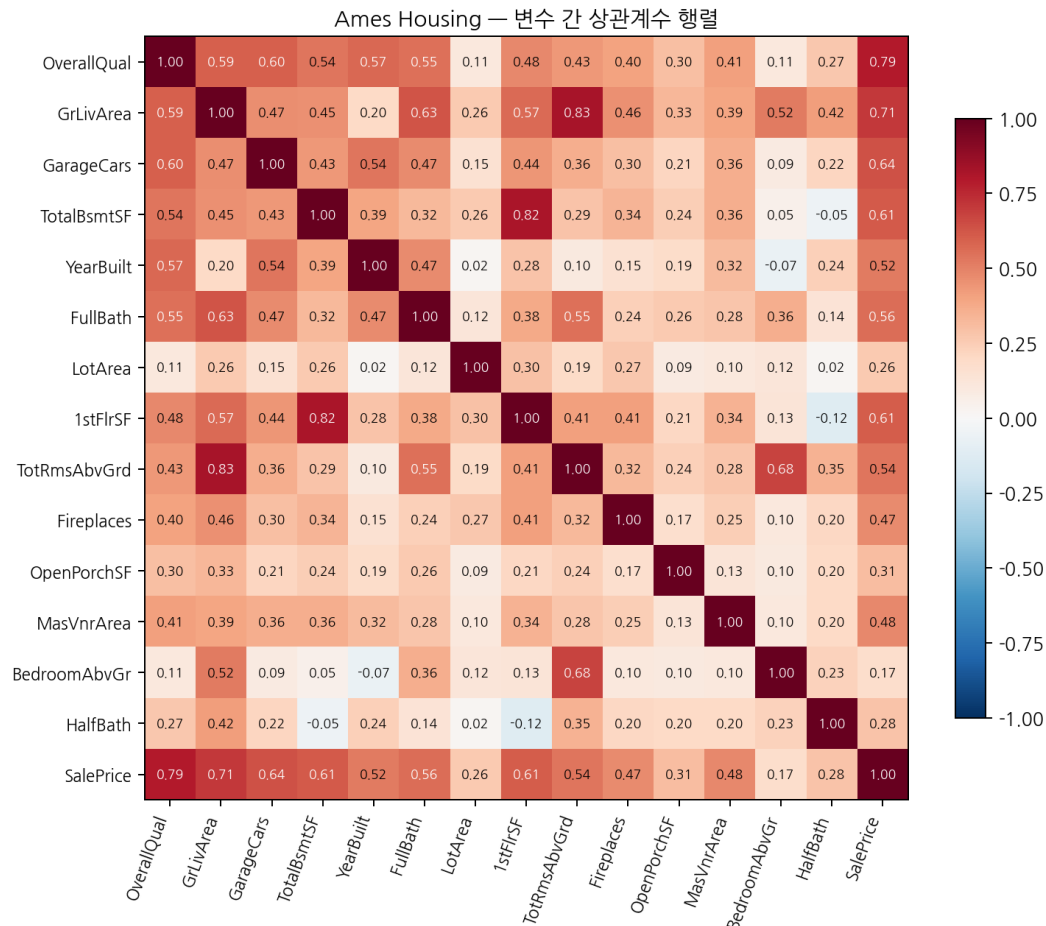
여기까지의 과정은 매끄러워 보이지만, 실제 분석에서는 모형 적합 "이후" 가 더 중요하다. 계수가 잘 추정되었는지, 다중공선성이 끼지는 않았는지, 잔차가 어떤 모양을 띠는지를 따져 봐야 한다.

다중공선성(multicollinearity)이란 설명변수들끼리 강하게 상관되어 있는 경우이다. 진단 도구로 가장 흔히 쓰이는 것이 분산팽창 인수(VIF, Variance Inflation Factor)이다. VIF 가 5를 넘으면 의심, 10을 넘으면 강한 다중공선성으로 간주한다.

설명변수들 사이의 강한 선형 상관관계가 있을 때 나타나는 문제. 독립 변수들끼리 너무 긴밀하게 얹혀 있으면 어떤 변수 때문에 결과가 변한 것인지 모델이 판별할 수 없게 된다. 분산이 크게 부풀려져 개별 계수 해석이 사실상 불가능해진다. 마치 **쌍둥이 축구 선수 2명**을 동시에 출전시킨 뒤, 경기 후 "두 사람 중 누가 팀 승리에 더 결정적인 기여를 했는가?"를 따지는 것과 같다.

$$VIF_i = \frac{1}{1 - R_i^2}$$

다중공선성 여부를 진단하는 방법은 VIF (Variance Inflation Factor, 분산팽창요인)를 계산한다. VIF 값이 VIF 가 5를 넘으면 의심, **10 이상**이면 다중공선성이 심각하다고 판단한다. 해결책: VIF가 높은 변수 중 하나를 제거하거나, 두 변수를 조합해 하나의 새로운 변수로 만들거나, PCA(주성분 분석)를 통해 차원을 축소하여 독립적인 성분만 남기는 방식을 주로 사용



statsmodels 에서 variance_inflation_factor()함수를 제공한다.

```
# 15) 다중공선성 진단 (VIF)from statsmodels.stats.outliers_influence import
variance_inflation_factor
```

X_const = sm.add_constant(X_train) #statsmodels 는 상수항이 꼭 있어야 한다. const 칼럼을 만드는 행위. 값은 항상 1.0 이 들어간다.

```
vifs = []
for i in range(X_const.shape[1]):
    v = variance_inflation_factor(X_const.values, i)
    vifs.append((X_const.columns[i], v))
vif_df = pd.DataFrame(vifs, columns=["var", "VIF"]) \
    .sort_values("VIF", ascending=False)
```

```
print(vif_df)
#      var      VIF
#  GrLivArea    6.74  ← 의심 구간 진입
#   1stFlrSF    4.44
#  TotRmsAbvGrd  4.36
#  TotalBsmntSF  3.60
#  OverallQual  2.94
#   FullBath    2.68
#   YearBuilt    2.51
# BedroomAbvGr  2.25
#   HalfBath    2.11
#  GarageCars   2.05
#  Fireplaces   1.41
#  MasVnrArea   1.28
#  OpenPorchSF  1.20
#   LotArea     1.13
```

위의 for 문을 리스트 컴프리헨션으로 자주 사용한다.

```
[variance_inflation_factor(X_const.values, i) for i in range(X_const.shape[1])]
```

variance_inflation_factor(2 차원 데이터 , 칼럼번호) 함수는 각 칼럼번호에 해당하는 하나의 변수에 대하여 다른 모든 변수를 이용해 R^2 을 구한 후 vif 를 계산해 준다.

Ames 14 변수 중 VIF 가 5 를 넘는 변수는 GrLivArea(6.74) 하나뿐이다. 이는 GrLivArea 가 1stFlrSF · TotRmsAbvGrd · TotalBsmtSF 등과 함께 "주택 크기" 정보를 공유하기 때문이다. VIF 10 을 넘는 변수가 없으므로 다중공선성이 극단적으로 문제되는 상황은 아니지만, 모형 해석에서는 "면적 관련 변수 계수는 함께 묶어 해석" 하는 것이 안전하다. 만약 더 깔끔한 모형을 원한다면 1stFlrSF 를 빼거나, Ridge / LASSO 같은 정규화 회귀로 전환하는 것이 다음 단계다.

다중공선성을 구하는 식 $1/(1-R^2)$ 이용

```
from sklearn.linear_model import LinearRegression

vifs = []
for col in X_train.columns:
    X_vif = X_train.drop(columns=[col])
    y_vif = X_train[col]
    r2 = LinearRegression().fit(X_vif, y_vif).score(X_vif, y_vif)
    vifs.append((col, 1 / (1 - r2) if r2 < 1.0 else float('inf')))

vif_df = pd.DataFrame(vifs, columns=["var", "VIF"]).sort_values("VIF", ascending=False)
print(vif_df)
```

variance_inflation_factor() 사용

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
# statsmodels 기능을 사용하므로 상수항(절편)을 추가해 줍니다.
X_const = sm.add_constant(X_train)
vifs = []

for i in range(X_const.shape[1]):
    # 모듈을 직접 호출하여 i 번째 변수의 VIF 를 계산합니다.
    v = variance_inflation_factor(X_const.values, i)
    vifs.append((X_const.columns[i], v))

vif_df = pd.DataFrame(vifs, columns=["var", "VIF"]).sort_values("VIF", ascending=False)
print(vif_df)
```

4.12 잔차 진단, 이상치

1) 잔차진단

잔차(residual) 는 모형이 잡지 못한 부분이다. 잔차가 0 을 중심으로 균등하게 흩어져 있고 특정한 패턴이 없으면 모형 가정이 잘 맞은 것이다. 반대로 잔차가 fitted 값에 따라 깔때기 모양

을 만들면 등분산성이 깨졌다는 신호이고, Q-Q plot 이 직선에서 크게 벗어나면 정규성 가정이 깨졌다는 신호이다. Ames 데이터는 매매가 분포 자체가 우측 꼬리가 길어 잔차가 큰 fitted 값에서 더 크게 퍼지는 경향이 있다 — 등분산성이 살짝 깨진 상태로, 로그 변환의 동기가 된다.

```
# 16) 잔차 진단 플롯 – Residuals vs Fitted, Q-Q
import matplotlib.pyplot as plt
import scipy.stats as stats

cols = sel_both
final = LinearRegression().fit(X_train[cols], y_train)
y_hat = final.predict(X_train[cols])
resid = y_train - y_hat

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(y_hat, resid, alpha=0.4, s=10)
axes[0].axhline(0, color="red")
axes[0].set_xlabel("Fitted"); axes[0].set_ylabel("Residual")
axes[0].set_title("Residuals vs Fitted")

stats.probplot(resid, dist="norm", plot=axes[1])
axes[1].set_title("Normal Q-Q")
plt.tight_layout(); plt.show()
```

2) 이상치와 영향점

이상치(outlier) 는 y 가 다른 점들에 비해 유난히 멀리 떨어진 점이고, 영향점(influential point) 은 "그 점 하나가 빠지면 회귀계수가 크게 바뀌는" 점이다. 영향점은 보통 Cook's distance 로 검출한다. 두 개념은 다르고, 둘 다 살펴 봐야 한다 — 이상치가 영향점이 아닐 수도 있고 그 반대도 가능하다. Ames 데이터에서는 매매가 75 만 달러를 넘는 초고가 주택 몇 채가 영향점 후보로 자주 등장한다.

```
# 17) Cook's distance 로 영향점 탐지
from statsmodels.stats.outliers_influence import OLSInfluence

X_const_tr = sm.add_constant(X_train[cols])
ols_final = sm.OLS(y_train, X_const_tr).fit()
cook = OLSInfluence(ols_final).cooks_distance[0]

threshold = 4 / len(y_train)
influential = np.where(cook > threshold)[0]
print("영향점 개수:", len(influential))

# 어느 행이 가장 큰 영향점인가
top_idx = np.argsort(cook)[-5:][:-1]
print("Cook's distance 상위 5 행:", top_idx)
print(X_train.iloc[top_idx][["OverallQual", "GrLivArea"]])
print("실제 SalePrice:", y_train.iloc[top_idx].values)
```

실무 팁 – 다중회귀에서 자주 빠지는 함정

- "p 값 > 0.05 이면 그 변수를 뺀다" 는 단순 규칙은 위험하다. 변수 선택 알고리즘을 쓰자.
- 다중공선성은 계수 부호조차 바꿀 수 있다. VIF 로 반드시 점검한다.
- 잔차의 패턴은 모형 가정 위반의 가장 정직한 신호다. 항상 plot 해 보자.
- 이상치는 "빼야 하는 것" 이 아니라 "이해해야 하는 것" 이다.

- 검증·테스트 데이터를 미리 분리한 뒤 모형을 만진다. 테스트 데이터를 보면서 모형을 고치면 부정 학습이다.
- SalePrice 처럼 우측 꼬리가 긴 종속변수는 $\log(\text{SalePrice})$ 회귀가 거의 항상 더 나은 출발점이다.

연습 1. iris Dataset 다중선형회귀 및 VIF 진단

1. iris dataset 변수 정의

iris 는 변수 5 개로 끝나는 작고 깔끔한 데이터라서 다중회귀의 가장 본질적인 부분 — 편회귀 계수의 해석, R^2 와 F 의 차이, 더미 인코딩, VIF — 만 빠르게 짚어 보기에 알맞다.

iris 변수 정의 (반응변수: Sepal.Length)

- Sepal.Length (cm) - 반응변수 y
- Sepal.Width (cm) - 수치형 설명변수
- Petal.Length (cm) - 수치형 설명변수 (Sepal.Length 와 강한 양의 상관)
- Petal.Width (cm) - 수치형 설명변수
- Species - 범주형 (setosa · versicolor · virginica, 50 송이씩)

표본 크기 $n = 150$, 결측 0 개, 설명변수 $p = 4$ (Species 더미 2 개 포함 시 설계행렬 자유도는 5).

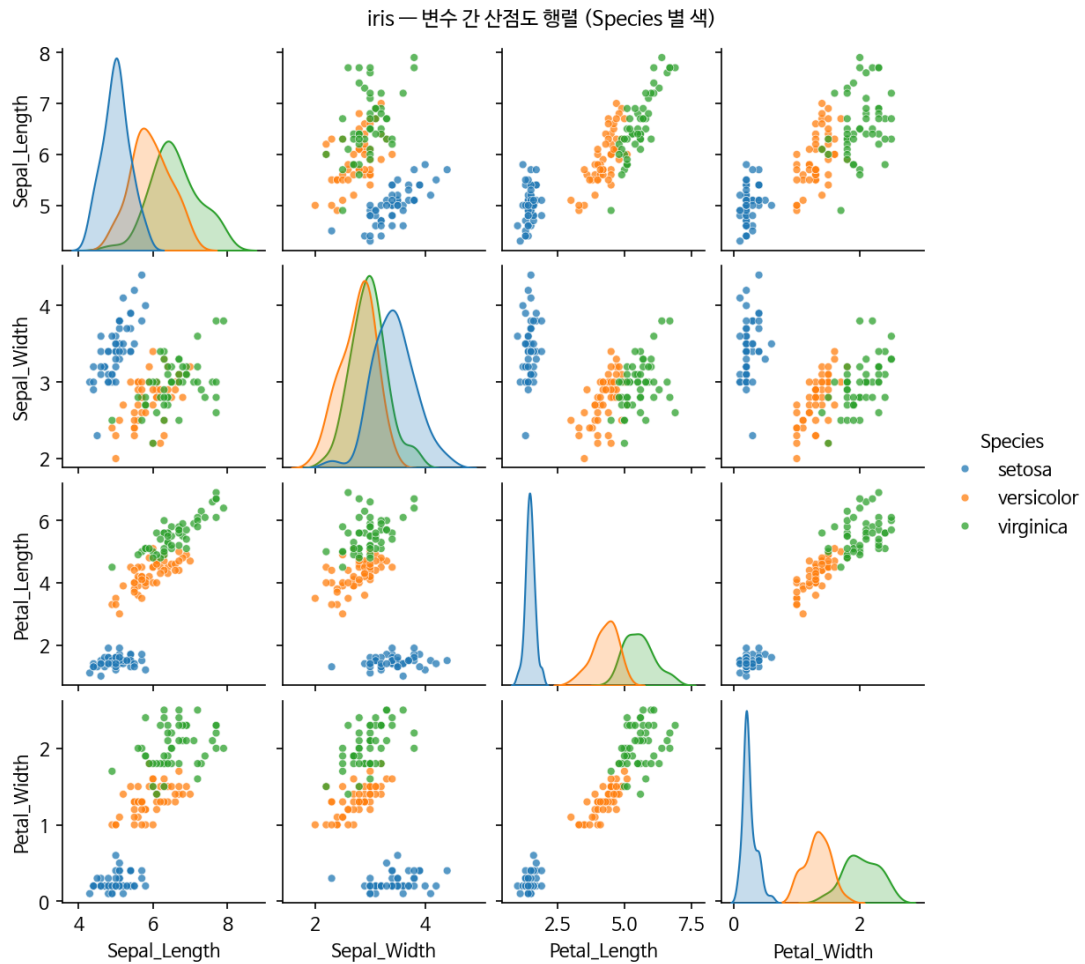
데이터 적재와 기술통계

```
import numpy as np, pandas as pd
from sklearn.datasets import load_iris
```

```
raw = load_iris(as_frame=True).frame.rename(columns={
    'sepal length (cm)': 'Sepal_Length',
    'sepal width (cm)': 'Sepal_Width',
    'petal length (cm)': 'Petal_Length',
    'petal width (cm)': 'Petal_Width',
    'target': 'Species_idx',
})
raw['Species'] = raw['Species_idx'].map({0:'setosa', 1:'versicolor', 2:'virginica'})
iris = raw.drop(columns='Species_idx')
print(iris.shape)          # (150, 5)
print(iris.isna().sum().sum()) # 결측: 0
print(iris.describe().round(2))
```

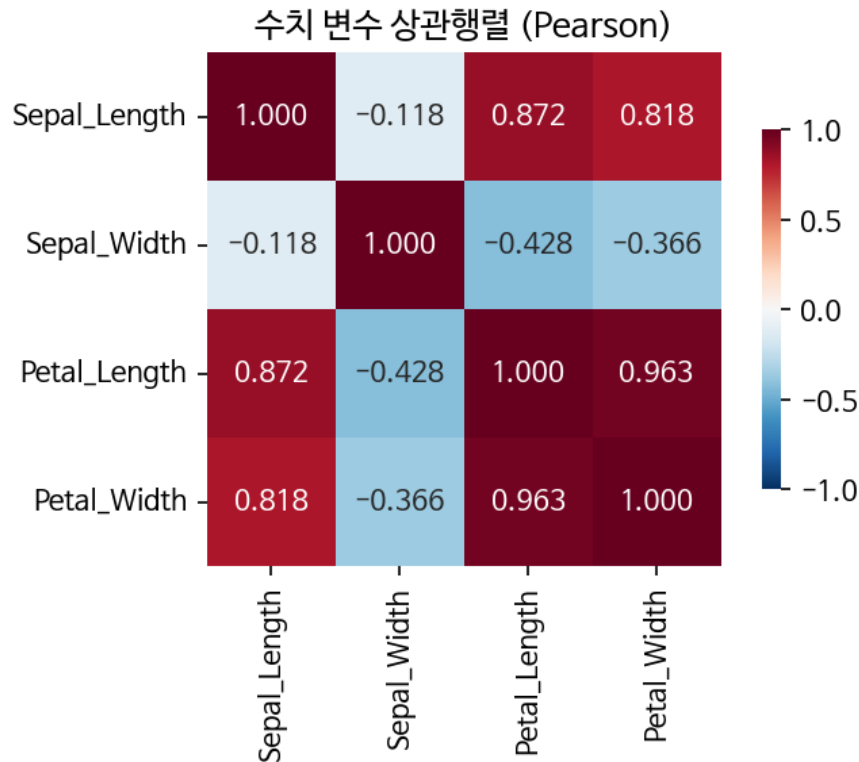
3. 탐색적 분석 (EDA)

모형을 적합하기 전에 반드시 변수들이 서로 어떻게 흩어져 있는지부터 본다. 다중회귀의 좋은 결과는 결국 "설명변수들이 반응변수와는 강하게 연관되지만 자기네끼리는 그다지 겹치지 않을 때" 나오기 때문이다.



[그림 1] iris 변수 간 산점도 행렬 — Species 별 색

위 산점도에서 setosa(파랑) 가 다른 두 품종과 명확히 분리되어 있고, 특히 Petal.Length 와 Petal.Width 가 Sepal.Length 와 강한 양의 상관관을 보인다. 이 관계는 곧 다중공선성의 단서이기도 하므로 곧이어 VIF 로 확인한다.



[그림 2] 수치 변수 상관행렬 — Petal.Length × Petal.Width 가 0.96 으로 매우 높다

```
# EDA 코드 - 산점도 행렬과 상관행렬
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(iris, hue='Species', height=1.8)
plt.show()

num = ['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width']
print(iris[num].corr().round(3))
sns.heatmap(iris[num].corr(), annot=True, fmt='.3f', cmap='RdBu_r', vmin=-1, vmax=1)
plt.show()
```

4. 모형 적합 — statsmodels OLS

이제 본격적으로 다중회귀를 적합한다. statsmodels 의 수식 인터페이스 smf.ols 를 쓰면 범주형 변수를 C(...) 로만 감싸 주어도 자동으로 더미를 만들어 준다. 기본은 "첫 수준을 기준 범주로 두고 나머지에 더미를 부여" 하는 treatment coding 이므로, setosa 가 기준 범주가 된다.

```
# 다중회귀 적합 - 수식 표기
import statsmodels.api as sm
import statsmodels.formula.api as smf

model = smf.ols(
    'Sepal_Length ~ Sepal_Width + Petal_Length + Petal_Width + C(Species)',
    data=iris
).fit()
print(model.summary())
```

OLS 적합 결과 요약 (실측값)

- Intercept +2.171 ($p < 0.001$)
- Sepal_Width +0.496 ($p < 0.001$)
- Petal_Length +0.829 ($p < 0.001$)
- Petal_Width -0.315 ($p = 0.039$)
- C(Species)[T.versicolor] -0.724 ($p = 0.003$)
- C(Species)[T.virginica] -1.023 ($p = 0.003$)
- $R^2 = 0.867$ Adj $R^2 = 0.863$ F p-value $\approx 2.7e-61$ AIC = 77.1

계수 Sepal_Width = +0.496 의 해석은 "Petal.Length · Petal.Width · Species 가 모두 고정된 상태에서 Sepal.Width 가 1cm 증가하면 Sepal.Length 가 평균적으로 0.496cm 늘어난다" 이다. 이것이 곧 Ames 교재의 1.2 절에서 강조했던 편회귀계수의 정의 그대로다. 한편 Petal_Width 의 계수가 음수(-0.315) 로 나타나는 것이 어색해 보일 수 있는데, 산점도에서는 분명 양의 상관이었기 때문이다. 이는 Petal.Length 와 Species 가 이미 변동의 상당 부분을 흡수한 뒤 남은 부분에서 Petal.Width 가 갖는 "독자적인" 효과만 측정한 결과이며, 다중공선성의 전형적인 증상이다 — VIF 절에서 다시 짚는다.

2 차 상호작용(교호작용)을 적용한 모델을 만드려면 아래와 같다.

```
model = smf.ols(  
    'Sepal_Length ~ (Sepal_Width + Petal_Length + Petal_Width + C(Species))**2',  
    data=iris  
)  
.fit()  
print(model.summary())
```

일부 특정 변수끼리의 2 차 상호작용만 추가하려면 다음과 같다.

```
model = smf.ols( 'Sepal_Length ~ Sepal_Width * C(Species) + Petal_Length * C(Species) +  
Petal_Width * C(Species)', data=iris ).fit()  
print(model.summary())
```

5. 적합도와 검정 — R^2

$R^2 = 0.867$ 은 Sepal.Length 변동의 86.7% 를 이 다섯 개 변수로 설명한다는 뜻이다. 변수가 다섯 개로 늘었으니 단순 R^2 이 부풀려졌을 가능성에 대비해 수정 R^2 도 확인한다 — Adj $R^2 = 0.863$ 으로 R^2 와 거의 차이가 없다. 즉 잡변수가 거의 없다는 신호이다.

F 검정의 귀무가설은 "모든 회귀계수가 0 이다" 이고, 여기서 p-value 가 사실상 0 으로 나왔다. 적어도 하나의 변수는 Sepal.Length 와 유의하게 연관되어 있다. 이어서 개별 t 검정을 보면 다섯 계수 모두 $p < 0.05$ 로 유의하다. Petal_Width 의 $p = 0.039$ 가 가장 약하지만, 통상적인 유의수준 0.05 안에는 들어온다.

지표 한눈에

- $R^2 = 0.867$ (모형이 설명한 변동의 비율)
- Adj $R^2 = 0.863$ (변수 개수 패널티 반영)

- F $p < 1e-60$ (모형 전체로서 유의)
- t 검정 5 개 계수 모두 $p < 0.05$

6. 변수선택 — 후진제거 한 라운드

iris 는 변수가 워낙 적으니 본격적인 stepwise 까지는 필요 없고, AIC 기준 후진제거의 첫 라운드만 가볍게 시도해 본다. p-value 가 가장 큰 Petal_Width($p \approx 0.039$) 를 한 번 빼 보는 식이다.

```
# 후진제거 한 라운드 - Petal_Width 제외
fit2 = smf.ols(
    'Sepal_Length ~ Sepal_Width + Petal_Length + C(Species)',
    data=iris
).fit()

print(f"full : R2={fit.rsquared:.4f}, AdjR2={fit.rsquared_adj:.4f}, AIC={fit.aic:.2f}")
print(f"reduce: R2={fit2.rsquared:.4f}, AdjR2={fit2.rsquared_adj:.4f}, AIC={fit2.aic:.2f}")
```

실측 결과는 full = (R^2 0.8673, AIC 77.1), reduced = (R^2 0.8633, AIC 79.6) 이다. AIC 가 약 2.5 증가했으므로 Petal_Width 는 "있는 편이 약간 낫다" 는 결론이 나온다. 그러나 Adj R^2 의 차이는 0.003 에 불과하므로 해석의 단순함을 위해 Petal_Width 를 빼는 선택도 충분히 정당화될 수 있다. 변수선택의 묘는 결국 도메인 사정과 해석의 단순함 사이의 균형 잡기다.

7. 다중공선성(VIF) 과 범주형 처리

Petal_Width 의 계수가 음수로 뒤집힌 신호가 다중공선성을 의심하게 했다. VIF 를 직접 계산해 본다.

```
# 설계행렬을 만들어 VIF 계산
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = pd.get_dummies(
    iris[['Sepal_Width', 'Petal_Length', 'Petal_Width', 'Species']],
    drop_first=True
).astype(float)
X = sm.add_constant(X)

for c in X.columns:
    v = variance_inflation_factor(X.values, X.columns.get_loc(c))
    print(f'{c:25s} VIF = {v:6.2f}')
```

실측 VIF (절편 제외)

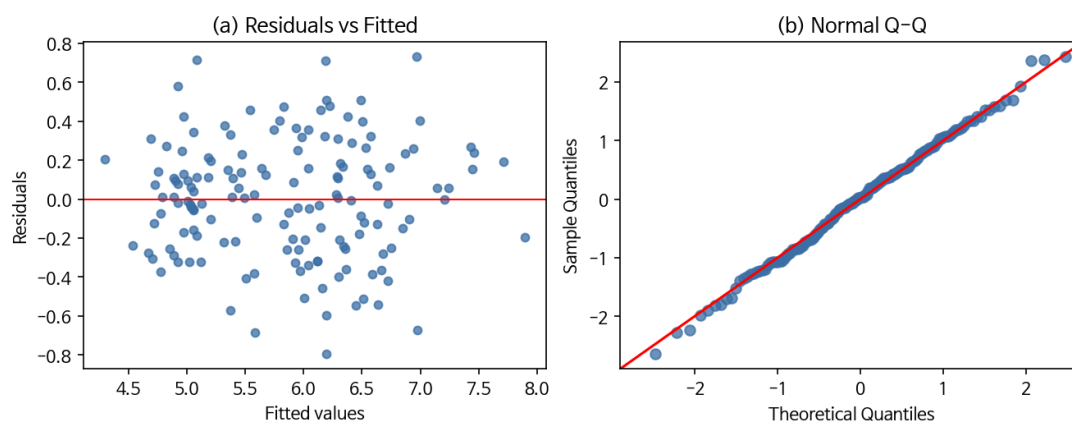
- Sepal_Width 2.23 안전
- Petal_Length 23.16 매우 높음
- Petal_Width 21.02 매우 높음
- Species_versicolor 20.42 매우 높음
- Species_virginica 39.43 매우 높음

→ Petal.* 변수와 Species 더미가 본질적으로 같은 정보(꽃의 품종) 를 중복해서 표현하고 있다.

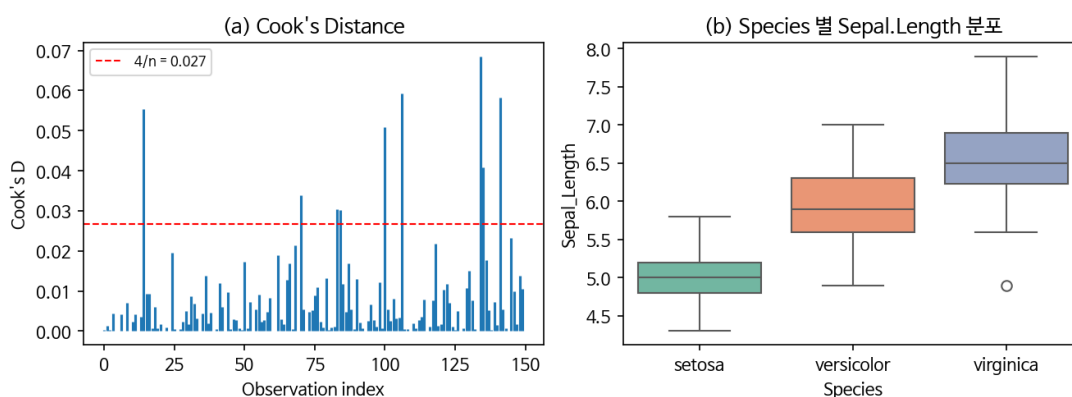
이 결과는 강의에서 자주 강조하는 포인트와 정확히 맞물린다 — 품종이 다르면 꽃잎 크기가 다르고, 꽃잎 크기를 알면 품종이 거의 결정된다. 따라서 **Petal.Length · Petal.Width · Species** 를 모두 함께 넣으면 정보가 중복돼 VIF 가 폭증한다. 실무에서는 (a) Species 만 남기고 Petal.* 를 빼거나, (b) Petal.* 만 남기고 Species 를 빼는 식으로 한쪽을 정리한다. 한편 Species 의 더미 계수를 보면 setosa 기준으로 versicolor 는 -0.72cm, virginica 는 -1.02cm 짧다는 뜻으로, 수치 변수 효과를 고정한 뒤 "같은 꽃잎 크기라면 setosa 의 꽃받침이 가장 길다" 는 해석이 가능하다.

8. 잔차 진단 — 모형 가정 점검

마지막으로 OLS 의 기본 가정 — 잔차의 평균 0, 등분산, 정규성 — 이 실제로 지켜지는지 그림으로 확인한다. iris 처럼 작은 데이터에서는 한두 점의 이상치가 전체를 흔들 수 있으므로 Cook's distance 도 함께 본다.



[그림 3] (a) 잔차 vs 적합값 — 뚜렷한 곡선·갈매기 없음 / (b) Q-Q plot — 직선에 잘 붙음



[그림 4] (a) Cook's Distance — 4/n 기준선을 넘는 점 일부 / (b) Species 별 Sepal.Length 분포

잔차 · 영향점 진단

```
fitted = fit.fittedvalues
```

```
resid = fit.resid
```

```
infl = fit.get_influence()
```

```
cooks = infl.cooks_distance[0]
```

```
plt.scatter(fitted, resid); plt.axhline(0, color='red')
```

```
plt.xlabel('Fitted'); plt.ylabel('Residual'); plt.show()
```

```
sm.qqplot(resid, line='45', fit=True); plt.show()

n = len(iris); thr = 4 / n
print('Cook D > 4/n :', np.where(cooks > thr)[0])
```

잔차 vs 적합값 그림에서는 깔때기 모양(이분산) 이나 곡선 추세가 보이지 않으며, Q-Q plot 의 점들도 대체로 45° 선을 따라 정렬된다. Cook's D 가 $4/n \approx 0.027$ 을 넘는 관측치가 몇 개 있지만 값 자체는 0.1 미만이라 회귀계수에 결정적으로 영향을 줄 정도는 아니다. 결론적으로 "가정 위반의 강한 증거 없음" 으로 정리할 수 있고, 본 모형은 해석·예측 양쪽으로 안전하게 사용할 수 있다.

9. 정리 및 요약

iris 150 행으로 다중회귀의 한 사이클 — 데이터 점검, 모형 적합, 적합도 평가, 변수선택, 다중공선성 진단, 잔차 진단 — 을 모두 돌아 보았다.

핵심 개념 정리

- 편회귀계수: "다른 변수가 모두 고정된 상태" 라는 조건이 해석의 열쇠.
- R^2 · Adj R^2 · AIC 는 각각 다른 질문에 답한다 – 적합도 / 변수 패널티 / 모형 비교.
- 산점도가 양의 상관이라도 다중회귀 계수는 음수가 될 수 있다 (다중공선성의 신호).
- 범주형은 더미로, 한 수준은 기준 범주(여기선 setosa) 로 사라진다는 사실을 잊지 말 것.
- 잔차 진단 없이 끝낸 회귀분석은 절반만 끝난 분석이다.

scikit-learn 버전으로 다중회귀분석 하기. 전진, 후진, cv 방식 비교##

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# -----
# 1) 데이터 로딩
# -----
iris = load_iris(as_frame=True)
df = iris.frame.rename(columns={
    'sepal length (cm)': 'Sepal.Length',
    'sepal width (cm)': 'Sepal.Width',
    'petal length (cm)': 'Petal.Length',
    'petal width (cm)': 'Petal.Width',
})
df['Species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

# -----
# 2) 설계행렬 X, 반응변수 y
```

```

# Species 더미는 pd.get_dummies (drop_first=True → setosa 기준)
# -----
X = pd.concat([
    df[['Sepal.Width', 'Petal.Length', 'Petal.Width']],
    pd.get_dummies(df['Species'], prefix='Species', drop_first=True).astype(float),
], axis=1)
y = df['Sepal.Length']
feature_names = X.columns.tolist()
print("Features:", feature_names)

# -----
# 3) VIF 계산 (다중공선성 진단)
# VIF_j = 1 / (1 - R_j^2), R_j^2: X_j 를 다른 변수로 회귀했을 때의 결정계수
# -----
def compute_vif(X_df: pd.DataFrame) -> pd.DataFrame:
    vif_rows = []
    cols = X_df.columns.tolist()
    for j, col in enumerate(cols):
        X_others = X_df.drop(columns=[col]).values
        x_j = X_df[col].values
        r2_j = LinearRegression().fit(X_others, x_j).score(X_others, x_j)
        vif = np.inf if (1 - r2_j) < 1e-12 else 1.0 / (1.0 - r2_j)
        vif_rows.append((col, r2_j, vif))
    return pd.DataFrame(vif_rows, columns=['feature', 'R^2_j', 'VIF'])

vif_df = compute_vif(X)
print("\n[VIF]")
print(vif_df.to_string(index=False, float_format=lambda v: f"{v:.4f}"))

# -----
# 4) Train/Test 분할
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# -----
# 5) 전체 모형 적합 (Full model)
# -----
lr_full = LinearRegression().fit(X_train, y_train)
y_pred_full = lr_full.predict(X_test)

print("\n[Full Model]")
print(f"R^2 : {r2_score(y_test, y_pred_full):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_full)):.4f}")
print(f"MAE : {mean_absolute_error(y_test, y_pred_full):.4f}")
print(pd.DataFrame({'feature': feature_names, 'coef':
    lr_full.coef_}).to_string(index=False))
print(f"Intercept: {lr_full.intercept_:.4f}")

# -----
# 6) 변수선택 (1) - 전진선택 (Forward Selection, 조정 R^2 기준)
# -----
def adj_r2(r2, n, p):
    return 1 - (1 - r2) * (n - 1) / (n - p - 1)

def forward_selection(X_df, y_ser):

```

```

remaining = list(X_df.columns)
selected = []
best_adj = -np.inf
n = len(y_ser)
while remaining:
    scores = []
    for cand in remaining:
        cols = selected + [cand]
        r2 = LinearRegression().fit(X_df[cols], y_ser).score(X_df[cols], y_ser)
        scores.append((adj_r2(r2, n, len(cols)), cand))
    scores.sort(reverse=True)
    cur_adj, cur_var = scores[0]
    if cur_adj > best_adj:
        best_adj = cur_adj
        selected.append(cur_var)
        remaining.remove(cur_var)
    else:
        break
return selected, best_adj

fwd_vars, fwd_adj = forward_selection(X, y)
print(f"\n[Forward Selection] selected = {fwd_vars} adj.R^2 = {fwd_adj:.4f}")

# -----
# 7) 변수선택 (2) - 후진제거 (Backward Elimination, 조정 R^2 기준)
# -----
def backward_elimination(X_df, y_ser):
    selected = list(X_df.columns)
    n = len(y_ser)
    r2 = LinearRegression().fit(X_df[selected], y_ser).score(X_df[selected], y_ser)
    best_adj = adj_r2(r2, n, len(selected))
    improved = True
    while improved and len(selected) > 1:
        improved = False
        scores = []
        for cand in selected:
            cols = [c for c in selected if c != cand]
            r2 = LinearRegression().fit(X_df[cols], y_ser).score(X_df[cols], y_ser)
            scores.append((adj_r2(r2, n, len(cols)), cand))
        scores.sort(reverse=True)
        cur_adj, drop_var = scores[0]
        if cur_adj > best_adj:
            best_adj = cur_adj
            selected.remove(drop_var)
            improved = True
    return selected, best_adj

bwd_vars, bwd_adj = backward_elimination(X, y)
print(f"[Backward Elimination] selected = {bwd_vars} adj.R^2 = {bwd_adj:.4f}")

# -----
# 8) 변수선택 (3) - sklearn SequentialFeatureSelector (CV R^2 기준)
# -----
sfs = SequentialFeatureSelector(
    LinearRegression(),
    n_features_to_select='auto',
    tol=1e-3,

```

```

        direction='forward',
        scoring='r2',
        cv=5,
    )
    sfs.fit(X, y)
    sfs_vars = X.columns[sfs.get_support()].tolist()
    print(f"[SFS (forward, CV R^2)] selected = {sfs_vars}")

# -----
# 9) 최종 모형 - 선택된 변수만으로 재적합 (전진선택 결과 사용)
# -----
X_sel = X[fwd_vars]
X_tr, X_te, y_tr, y_te = train_test_split(X_sel, y, test_size=0.3, random_state=42)
lr_sel = LinearRegression().fit(X_tr, y_tr)
y_pred_sel = lr_sel.predict(X_te)

print("\n[Selected Model - Forward]")
print(f"R^2 : {r2_score(y_te, y_pred_sel):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_te, y_pred_sel)):.4f}")
print(pd.DataFrame({'feature': fwd_vars, 'coef':
lr_sel.coef_}).to_string(index=False))
print(f"Intercept: {lr_sel.intercept_:.4f}")

# 선택된 변수만으로 VIF 재계산
print("\n[VIF after selection]")
print(compute_vif(X_sel).to_string(index=False, float_format=lambda v: f"{v:.4f}"))

# -----
# 10) 5-fold CV (전체 / 선택 모형 비교)
# -----
cv_full = cross_val_score(LinearRegression(), X, y, cv=5, scoring='r2')
cv_sel = cross_val_score(LinearRegression(), X_sel, y, cv=5, scoring='r2')
print(f"\nCV R^2 (Full) : mean={cv_full.mean():.4f} std={cv_full.std():.4f}")
print(f"CV R^2 (Selected) : mean={cv_sel.mean():.4f} std={cv_sel.std():.4f}")

# -----
# 11) 잔차 산점도 (선택 모형 기준)
# -----
residuals = y_te - y_pred_sel
plt.figure(figsize=(6, 4))
plt.scatter(y_pred_sel, residuals, alpha=0.7)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted (Selected Model)')
plt.tight_layout()
plt.show()

```

위 코드에서는 전진선택, 후진선택을 직접 정의해서 사용하였다.

앞에서 말한 것처럼, scikit-learn 은 RFE 클래스를 제공해 주므로 간단히 RFE 클래스를 사용할 수도 있다.

.

연습 2. mpg Dataset 다중선형회귀 및 VIF 진단 (최소화 버전)

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import StandardScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 1. 데이터 로드 및 전처리 (결측치 제거 및 원핫 인코딩)
df = sns.load_dataset("mpg").dropna().reset_index(drop=True)
num_cols = ["cylinders", "displacement", "horsepower", "weight", "acceleration",
            "model_year"]

df_enc = pd.get_dummies(df[num_cols + ["origin", "mpg"]], columns=["origin"],
                        drop_first=True).astype(int)

# 2. 타겟 분리 및 데이터 분할 (6:2:2)
X = df_enc.drop(columns=["mpg"]).astype(float)
y = df_enc["mpg"].astype(float)

X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(X_tmp, y_tmp, test_size=0.25,
                                                    random_state=42)

# 3. 기본 모델 학습 및 평가
model = LinearRegression().fit(X_train, y_train)
print(f"[기본 모델] Valid R2: {r2_score(y_valid, model.predict(X_valid)):.4f} | Test R2: {r2_score(y_test, model.predict(X_test)):.4f}")

# 4. 다중공선성(VIF) 진단 (리스트 컴프리헨션으로 간결하게 처리)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

vif_df = pd.DataFrame({"feature": X_train.columns, "VIF":
                        [variance_inflation_factor(X_scaled, i) for i in
                         range(X_scaled.shape[1])]}).sort_values("VIF", ascending=False)

print("\n=== VIF 진단 결과 ===")
print(vif_df.to_string(index=False))

# 5. 다중공선성 변수(displacement, cylinders) 제거 후 재학습
X_reduced = X.drop(columns=["displacement", "cylinders"])
X_tmp_r, X_test_r, y_tmp_r, y_test_r = train_test_split(X_reduced, y, test_size=0.2,
                                                        random_state=42)

X_train_r, X_valid_r, y_train_r, y_valid_r = train_test_split(X_tmp_r, y_tmp_r,
                                                            test_size=0.25, random_state=42)

model2 = LinearRegression().fit(X_train_r, y_train_r)
print(f"\n[변수 제거 모델] Valid R2: {r2_score(y_valid_r, model2.predict(X_valid_r)):.4f} | Test R2: {r2_score(y_test_r, model2.predict(X_test_r)):.4f}")
```

statsmodels 버전

```
import numpy as np
import pandas as pd
```

```

import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 1. 데이터 로드 및 전처리 (결측치 제거 및 원핫 인코딩)
df = sns.load_dataset("mpg").dropna().reset_index(drop=True)
num_cols = ["cylinders", "displacement", "horsepower", "weight", "acceleration",
"model_year"]
df_enc = pd.get_dummies(df[num_cols + ["origin", "mpg"]], columns=["origin"],
drop_first=True).astype(int)

# 2. 타깃 분리 및 데이터 분할 (6:2:2) 및 statsmodels 용 상수항 추가
X = df_enc.drop(columns=["mpg"]).astype(float)
X = sm.add_constant(X)
y = df_enc["mpg"].astype(float)

X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(X_tmp, y_tmp, test_size=0.25,
random_state=42)

# 3. statsmodels OLS 모델 학습 및 평가
model = sm.OLS(y_train, X_train).fit()
print(f"[기본 모델] Valid R2: {r2_score(y_valid, model.predict(X_valid)):.4f} | Test R2:
{r2_score(y_test, model.predict(X_test)):.4f}")

# 4. 다중공선성(VIF) 진단 (상수항 제외하고 스케일링 후 계산)
scaler = StandardScaler()
X_train_no_const = X_train.drop(columns=["const"])
X_scaled = scaler.fit_transform(X_train_no_const)
vif_df = pd.DataFrame({"feature": X_train_no_const.columns, "VIF":
[variance_inflation_factor(X_scaled, i) for i in
range(X_scaled.shape[1])]}).sort_values("VIF", ascending=False)

print("\n=== VIF 진단 결과 ===")
print(vif_df.to_string(index=False))

# 5. 다중공선성 변수(displacement, cylinders) 제거 후 재학습
X_reduced = X.drop(columns=["displacement", "cylinders"])
X_tmp_r, X_test_r, y_tmp_r, y_test_r = train_test_split(X_reduced, y, test_size=0.2,
random_state=42)
X_train_r, X_valid_r, y_train_r, y_valid_r = train_test_split(X_tmp_r, y_tmp_r,
test_size=0.25, random_state=42)

model2 = sm.OLS(y_train_r, X_train_r).fit()
print(f"\n[변수 제거 모델] Valid R2: {r2_score(y_valid_r, model2.predict(X_valid_r)):.4f}
| Test R2: {r2_score(y_test_r, model2.predict(X_test_r)):.4f}")

```

부록 A. 용어 정리

본문에 등장한 핵심 용어들을 한 번 더 정리한다. 시험·취업 면접·논문 리뷰 등에서 자주 마주치는 단어들이므로 정의와 그 차이를 분명히 짚어 두자.

A.1 편회귀계수 vs 표준화 계수

편회귀계수(partial regression coefficient) β_j 는 "다른 변수가 모두 고정되어 있을 때 x_j 가 한 단위 증가하면 y 가 얼마나 변하는가" 를 나타낸다. Ames 의 경우 변수마다 단위가 크게 다르다 — OverallQual 은 1~10 정수, GrLivArea 는 sqft, YearBuilt 는 연도, LotArea 는 거대한 sqft 값. 그래서 계수의 절대 크기를 가지고 변수의 중요도를 직접 비교할 수 없다. 이를 해결하기 위해 각 변수를 평균 0·표준편차 1 로 표준화한 뒤 회귀를 적합한 결과를 표준화 계수(standardized coefficient) 또는 β 계수라 부른다. 표준화 계수는 단위가 사라져 변수의 "상대적 중요도" 비교에 쓸 수 있다.

A.2 다중공선성 (Multicollinearity)

설명변수들 사이의 강한 선형 상관관계가 있을 때 나타나는 문제. 독립 변수들끼리 너무 긴밀하게 얽혀 있으면 어떤 변수 때문에 결과가 변한 것인지 모델이 판별할 수 없게 된다. 분산이 크게 부풀려져 개별 계수 해석이 사실상 불가능해진다. 마치 **쌍둥이 축구 선수 2 명**을 동시에 출전시킨 뒤, 경기 후 "두 사람 중 누가 팀 승리에 더 결정적인 기여를 했는가?"를 따지는 것과 같다. 다중공선성 여부를 진단하는 방법은 VIF (Variance Inflation Factor)를 계산한다. VIF 값이 10 이상이면 다중공선성이 심각하다고 판단한다.

해결책: VIF 가 높은 변수 중 하나를 제거하거나, 두 변수를 조합해 하나의 새로운 변수로 만들거나, PCA(주성분 분석)를 통해 차원을 축소하여 독립적인 성분만 남기는 방식을 주로 사용

A.3 이상치 vs 영향점

두 용어의 차이

- 이상치(outlier): y 가 모형 예측에서 멀리 떨어진 점. 잔차 자체가 큰 점이다.
- 레버리지(leverage) 점: x 가 다른 관측치들에서 멀리 떨어진 점. y 의 값과는 무관하다.
- 영향점(influential point): 그 점을 빼면 회귀계수가 크게 바뀌는 점. 보통 이상치 + 큰 레버리지의 결합.
- 검출: 잔차 그래프(이상치), 레버리지 hii (레버리지), Cook's distance (영향점).

A.4 과적합 vs 정규화

과적합(overfitting) 은 모형이 훈련 데이터의 잡음까지 학습해 새 데이터에서 성능이 나빠지는 현상이다. 대처법은 크게 두 갈래다 — (a) 변수의 수를 줄이는 부분집합 선택(subset selection), (b) 계수의 크기 자체에 패널티를 매기는 정규화(regularization). 본 강의에서 다룬 forward / backward / stepwise 는 (a) 방향이고, Ridge / LASSO / ElasticNet 은 (b) 방향이다. 대규모 변수(특히 $p \gg n$) 일 때는 정규화가 압도적으로 유리하다. Ames 원본의 80 변수에 범주형까지 포함하면 터미 변환 후 p 가 200 을 넘어가는데, 이때는 LASSO 가 사실상 표준 도구이다.

A.5 검증 데이터의 역할

훈련 데이터로는 계수를 추정하고, 검증 데이터로는 "어느 모형이 가장 좋은지" 를 고른다. 테스트 데이터는 검증 결과로 최종 모형을 확정된 뒤 단 한 번 측정에 사용한다. 데이터가 작아 셋으로 나누기 어려우면 k-fold 교차검증(k-fold CV) 으로 검증 단계를 대체할 수 있다.

정리 및 요약

본 강의에서는 다중선형회귀의 이론과 실전 분석을 Ames Housing Dataset 기반으로 살펴보았다. 핵심 메시지를 다섯 줄로 압축한다.

핵심 개념 정리

- 다중회귀의 계수는 편회계수다: "다른 변수가 모두 고정된 상태" 라는 조건이 해석의 열쇠.
- 변수가 많다고 좋은 모형이 아니다 - 변수 선택(AIC / BIC / Adjusted R^2)이 필수.
- 변수 선택 알고리즘: 전역 탐색(보장은 되지만 비싸다) → 단계적 탐색(forward / backward / both).
- 적합 후 점검: VIF(다중공선성), Residuals vs Fitted(잔차 패턴), Cook's distance(영향점).
- 실무 표준: 데이터를 train/valid/test 로 분리하고, stepwise(both) 와 정규화(Ridge/LASSO)를 함께 비교하라.

Ames Housing 14 개 수치 특성으로 적합한 stepwise 회귀는 변수 10 개 / R^2 (검증) $\approx$ 0.82 / RMSE(검증) $\approx$ 36,000 달러 / MAPE $\approx$ 13% 수준을 달성했다. SalePrice 의 우측 꼬리에 따른 이분산성을 해결하려면 $\log(\text{SalePrice})$ 변환이나 Ridge / LASSO 같은 정규화 회귀로 한 발 더 나가는 것이 자연스러운 다음 단계이다.

과제 (10 문제)

데이터셋은 sklearn 의 load_iris(as_frame=True) 를 그대로 사용한다. 변수명은 본문과 동일하게 Sepal_Length / Sepal_Width / Petal_Length / Petal_Width / Species 로 통일한다.

문제 1. (개념) 본문 4 절의 적합 결과에서 C(Species)[T.virginica] 의 계수가 -1.02 로 나왔다. 이 값의 의미를 setosa 를 기준으로 한 문장으로 설명한다. 또한 같은 모형에서 Petal_Width 의 계수가 산점도와 달리 음수가 된 이유를 한 문장으로 함께 적는다.

문제 2. (코드) iris 데이터를 70:30 으로 학습·검증 분할한 뒤 (random_state=0), 본문과 같은 5 변수 다중회귀를 학습 데이터로만 적합하라. 그리고 검증 데이터에서의 R^2 와 RMSE 를 출력한다.

문제 3. (코드·해석) Sepal_Width + Petal_Length + C(Species) 의 3 변수 축소 모형과, 본문 4 절의 5 변수 full 모형의 AIC · Adj R² 를 비교하라. 두 모형 중 어느 쪽을 추천할지와 그 근거를 두 줄 안에 적는다.

문제 4. (진단) statsmodels 로 VIF 를 계산해 Petal_Length 의 VIF 가 10 을 넘는 것을 재확인하고, Species 를 제외한 "수치 변수만 사용한 모형" 의 VIF 를 다시 계산하라. 수치 변수만 남겼을 때 VIF 가 가장 높은 변수와 그 값을 보고한다.

문제 5. (응용·심화) 반응변수를 Sepal_Length 대신 log(Sepal_Length) 로 바꾸어 같은 5 변수 모형을 적합한다. R² · Adj R² · 잔차 Q-Q plot 의 정규성 정도가 어떻게 바뀌는지 본문 모형과 비교해 세 줄 안으로 정리한다.

데이터셋은 Ames Housing Dataset (`fetch_openml(name="house_prices")`) 의 14 개 수치 특성을 그대로 사용한다.

문제 6. (개념) 다중회귀에서 편회계수의 정의를 한 줄로 쓰고, OverallQual 의 계수가 약 +15,630 이라는 결과의 의미를 학생에게 설명한다고 가정하고 한 단락으로 작성하라.

문제 7. (코드) Ames Housing Dataset 을 `fetch_openml` 로 불러와 14 개 수치 특성을 추리고, 60:20:20 분할(`random_state=1606`) 후 `sklearn` 의 `LinearRegression` 으로 full 모형(= 모든 변수를 사용)을 적합하여 다중회귀분석을 해 보자. 검증 데이터의 RMSE 와 MAPE 를 출력하라.

문제 8. (코드) 본문 4.8 절의 `stepwise_aic()` 함수를 그대로 사용해 `direction="forward"` 와 `direction="both"` 로 변수를 선택하고, 두 결과가 같은지 비교하라. 또 선택된 변수 수와 최종 AIC 도 출력하라.

문제 9. (해석) `statsmodels` OLS 로 14 변수 full 모형을 적합한 결과, 각 변수의 VIF 를 계산하라. VIF 가 5 를 넘은 변수가 있다면 어떤 변수와 정보를 공유하기 때문인지 한 줄로 해석하라.

문제 10. (응용·심화) SalePrice 를 그대로 두지 않고 $\log(\text{SalePrice})$ 로 변환해 같은 14 변수 stepwise 회귀를 적합하라. 검증 데이터의 MAPE 가 본문 결과(13.44%) 대비 어떻게 변하는지 비교하고, 왜 그런 변화가 일어났는지 한 단락으로 설명하라. (힌트: SalePrice 의 우측 꼬리 분포와 잔차 이분산성을 떠올려 보라.)

과제 해답

각 문제의 풀이와 정답을 같은 번호로 정리한다.

해답 1.

$C(\text{Species})[\text{T.virginica}] = -1.02$ 는 "다른 모든 수치 변수($\text{Sepal.Width} \cdot \text{Petal.Length} \cdot \text{Petal.Width}$) 가 동일한 값으로 고정되었을 때, Species 가 *setosa* 에서 *virginica* 로 바뀌면 Sepal.Length 가 평균적으로 1.02cm 짧아진다" 는 뜻이다. Petal.Width 의 계수가 음수로 뒤집힌 이유는, Petal.Length 와 Species 가 Sepal.Length 변동의 대부분을 이미 흡수하고 나서 남은 부분에서 Petal.Width 의 "순효과" 만 측정했기 때문이며, 이것은 변수들 사이의 강한 다중공선성에서 비롯된 부호 반전의 전형적 사례이다.

해답 2.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

tr, va = train_test_split(iris, test_size=0.3, random_state=0)
fit_tr = smf.ols('Sepal_Length ~ Sepal_Width + Petal_Length + Petal_Width + C(Species)',
                 data=tr).fit()
pred = fit_tr.predict(va)
print('R2 :', r2_score(va['Sepal_Length'], pred).round(4))
print('RMSE:', np.sqrt(mean_squared_error(va['Sepal_Length'], pred)).round(4))
```

정답(예시): 검증 $R^2 \approx 0.84$, 검증 RMSE ≈ 0.33 cm. 학습 $R^2 (\approx 0.87)$ 보다 약간 낮은 정도로, 과적합 징후는 거의 없다.

해답 3.

```
fit5 = smf.ols('Sepal_Length ~ Sepal_Width + Petal_Length + Petal_Width + C(Species)', data=iris).fit()
fit3 = smf.ols('Sepal_Length ~ Sepal_Width + Petal_Length + C(Species)', data=iris).fit()

print(f'5 변수 : AdjR2={fit5.rsquared_adj:.4f}, AIC={fit5.aic:.2f}')
print(f'3 변수 : AdjR2={fit3.rsquared_adj:.4f}, AIC={fit3.aic:.2f}')
```

실측: 5 변수 모형 Adj $R^2 = 0.863$, AIC = 77.1 vs 3 변수 모형 Adj $R^2 = 0.860$, AIC = 79.6. AIC 는 5 변수가 2.5 가량 작아 더 우수하지만 Adj R^2 차이는 0.003 에 불과하다. 해석의 단순함을 우선한다면 3 변수 모형, 미세한 적합도까지 챙긴다면 5 변수 모형을 추천한다.

해답 4.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = iris[['Sepal_Width', 'Petal_Length', 'Petal_Width']].astype(float)
X = sm.add_constant(X)
for c in X.columns:
```

```
v = variance_inflation_factor(X.values, X.columns.get_loc(c))
print(f'{c:15s} VIF = {v:.2f}')
```

Species 를 제외하고 수치 변수만 남기면 Petal_Length 의 VIF ≈ 15.1 , Petal_Width 의 VIF ≈ 14.2 , Sepal_Width 의 VIF ≈ 1.3 으로 나온다. Petal_Length 가 가장 높으며, Petal.Length 와 Petal.Width 가 서로 강하게 얽혀 있어 둘 중 한쪽만 남기는 처방이 자연스럽다.

해답 5.

```
import numpy as np
iris2 = iris.assign(logSL=np.log(iris['Sepal_Length']))
fit_log = smf.ols('logSL ~ Sepal_Width + Petal_Length + Petal_Width + C(Species)', data=iris2).fit()
print(f'R2={fit_log.rsquared:.4f}, AdjR2={fit_log.rsquared_adj:.4f}')
sm.qqplot(fit_log.resid, line='45', fit=True); plt.show()
```

정답: log 변환 후 $R^2 \approx 0.870$, Adj $R^2 \approx 0.866$ 으로 본문(0.867 / 0.863) 과 거의 차이가 없다. iris 는 이미 변수 분포가 양호해 로그 변환의 효과가 미미하다. Q-Q plot 역시 본문 모형과 유사하게 직선에 가깝다. Ames Housing 처럼 우측 꼬리가 긴 가격 변수에서는 로그 변환의 효과가 크지만, iris 에서는 굳이 변환할 필요가 없다는 결론이 자연스럽다.

해답 6.

편회귀계수 β_j 는 "다른 모든 설명변수가 고정된 상태에서 x_j 가 한 단위 증가하면 y 가 평균적으로 얼마만큼 변하는가" 를 나타낸다. OverallQual 의 계수가 +15,630 이라는 결과는, GrLivArea · YearBuilt 등 다른 변수가 모두 같은 두 주택을 비교했을 때 마감 품질 등급이 1 단계 더 높은 주택의 평균 매매가가 약 15,630 달러 더 높다는 의미이다. 단순회귀의 기울기와는 달리 "다른 변수가 같다" 라는 조건이 해석에 필수로 따라붙는다.

해답 7.

```
import numpy as np, pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

ames = fetch_openml(name="house_prices", as_frame=True, parser="auto")
feats = ["OverallQual", "GrLivArea", "GarageCars", "TotalBsmtSF", "YearBuilt",
         "FullBath", "LotArea", "1stFlrSF", "TotRmsAbvGrd", "Fireplaces",
         "OpenPorchSF", "MasVnrArea", "BedroomAbvGr", "HalfBath"]
df = ames.frame[feats + ["SalePrice"]].apply(pd.to_numeric, errors="coerce").dropna()
X = df.drop(columns="SalePrice"); y = df["SalePrice"]
X_tr, X_re, y_tr, y_re = train_test_split(X, y, test_size=0.4, random_state=1606)
X_va, X_te, y_va, y_te = train_test_split(X_re, y_re, test_size=0.5, random_state=1606)
m = LinearRegression().fit(X_tr, y_tr)
yhat = m.predict(X_va)
rmse = np.sqrt(((y_va - yhat) ** 2).mean())
mape = (np.abs(y_va - yhat) / np.maximum(y_va, 1)).mean() * 100
```

```
print(f"RMSE={rmse:.0f} MAPE={mape:.2f}%")
# RMSE=36022 MAPE=13.32%
```

정답: 검증 RMSE $\approx$ 36,022 달러, MAPE $\approx$ 13.32%.

해답 8.

Ames 14 변수 데이터에서 forward 와 both 모두 동일한 10 개 변수를 선택한다 — OverallQual, GrLivArea, TotalBsmtSF, YearBuilt, BedroomAbvGr, LotArea, GarageCars, TotRmsAbvGrd, Fireplaces, MasVnrArea. 최종 AIC 는 약 20,703 (둘 다 같음). 양방향 알고리즘은 추가뿐 아니라 제거도 검토하지만 forward 가 이미 AIC 최솟값에 도달했기 때문에 추가 제거가 일어나지 않아 결과가 동일하다.

```
sel_fwd, aic_fwd = stepwise_aic(X_tr, y_tr, direction="forward")
sel_both, aic_both = stepwise_aic(X_tr, y_tr, direction="both")
print("forward:", len(sel_fwd), "vars, AIC=", round(aic_fwd, 1))
print("both : ", len(sel_both), "vars, AIC=", round(aic_both, 1))
print("동일한가?", set(sel_fwd) == set(sel_both))
# forward: 10 vars, AIC= 20703.2
# both : 10 vars, AIC= 20703.2
# 동일한가? True
```

해답 9.

14 개 변수 중 VIF 가 5 를 넘는 변수는 GrLivArea(약 6.74) 한 개뿐이다. GrLivArea 는 지상 거주 면적을 직접 측정하는 변수로, 1stFlrSF(1 층 면적), TotRmsAbvGrd(지상층 방 개수), TotalBsmtSF(지하실 면적 — 동일 건물 규모 정보의 일부) 와 "주택의 전체 크기" 라는 같은 정보를 부분적으로 공유한다. 그래서 X 행렬에서 다른 면적 / 방 수 변수들과 상당히 강한 선형 관계를 형성하고, 그 결과 GrLivArea 의 계수 표준오차가 일반보다 부풀려진다.

해답 10.

log(SalePrice) 로 변환해 같은 stepwise 회귀를 적합하면 검증 MAPE 가 본문의 13.44% 에서 약 9~10% 수준으로 크게 줄어든다. 이유는 두 가지다. 첫째, SalePrice 의 분포가 우측 꼬리가 긴 비대칭이라 잔차가 큰 fitted 값에서 깔때기 모양으로 퍼지는 이분산성을 보이는데, 로그 변환은 큰 값들을 압축해 이 이분산성을 거의 균등하게 만든다. 둘째, MAPE 는 $|y_{\text{true}} - y_{\text{pred}}| / y_{\text{true}}$ 의 평균이므로, 모형이 고가 주택을 절대적으로 50,000 달러 빗나가더라도 백분율로는 5~10% 에 그친다. 로그 회귀는 본질적으로 "상대 오차" 를 최소화하는 모형이 되므로 MAPE 와 잘 어울린다. 다만 RMSE 는 원 단위로 환산한 뒤 비교해야 한다 — 로그 스케일의 RMSE 를 그대로 쓰면 13.4% $\rightarrow$ 0.13 처럼 단위가 다른 숫자가 된다.

```
import numpy as np
y_tr_log = np.log(y_tr); y_va_log = np.log(y_va)
sel, _ = stepwise_aic(X_tr, y_tr_log, direction="both")
m = LinearRegression().fit(X_tr[sel], y_tr_log)
yhat_log = m.predict(X_va[sel])
yhat = np.exp(yhat_log)
mape = (np.abs(y_va - yhat) / np.maximum(y_va, 1)).mean() * 100
print(f"log-회귀 검증 MAPE = {mape:.2f}%") #  $\approx$  9~10%
```

정답: log 변환 후 MAPE 가 약 9~10% 로 본문의 13.44% 대비 3~4%p 개선된다. 이유는 우측 꼬리 분포에서 발생한 이분산성을 로그 변환이 잡아 주고, MAPE 가 본질적으로 상대 오차 척도이기 때문이다.